

Lecture-#19

Natural Language Processing

A8706

UNIT-V

Text Summarization and Information
Retrieval

Question Answering

- ❖ Question answering (QA) systems retrieve answers to user questions from information sources.
- ❖ QA systems attempt to provide answers that are accurate and to the point. Often, the web is utilized as a large, redundant, and up-to-date information source.
- ❖ However, local text corpora are frequently used for advanced search techniques that require the sources to be preprocessed, for restricted domains (e.g., medical, legal, or intranet data), and in evaluations that require results to be comparable and reproducible.
- ❖ Question types that have been best studied to date are **factoid questions**, which ask for concise answers such as named entities (e.g., *What is the capital of Turkey?*), and **list questions** seeking lists of such factoid answers (e.g., *Which countries are in NATO?*).

Question Answering

- ❖ Attempts have been made to tackle questions with complex answers, such as
 - definitional questions** requesting information on a given topic, including biographies for people (e.g., *Who is Albert Einstein?*),
 - relationship questions** (e.g., *What is the relationship between the Taliban and Al-Qaeda?*), and
 - opinion questions** (e.g., *What do people like about IKEA?*).

Question Answering



Learning from Experience, Sharing with Passion

History:

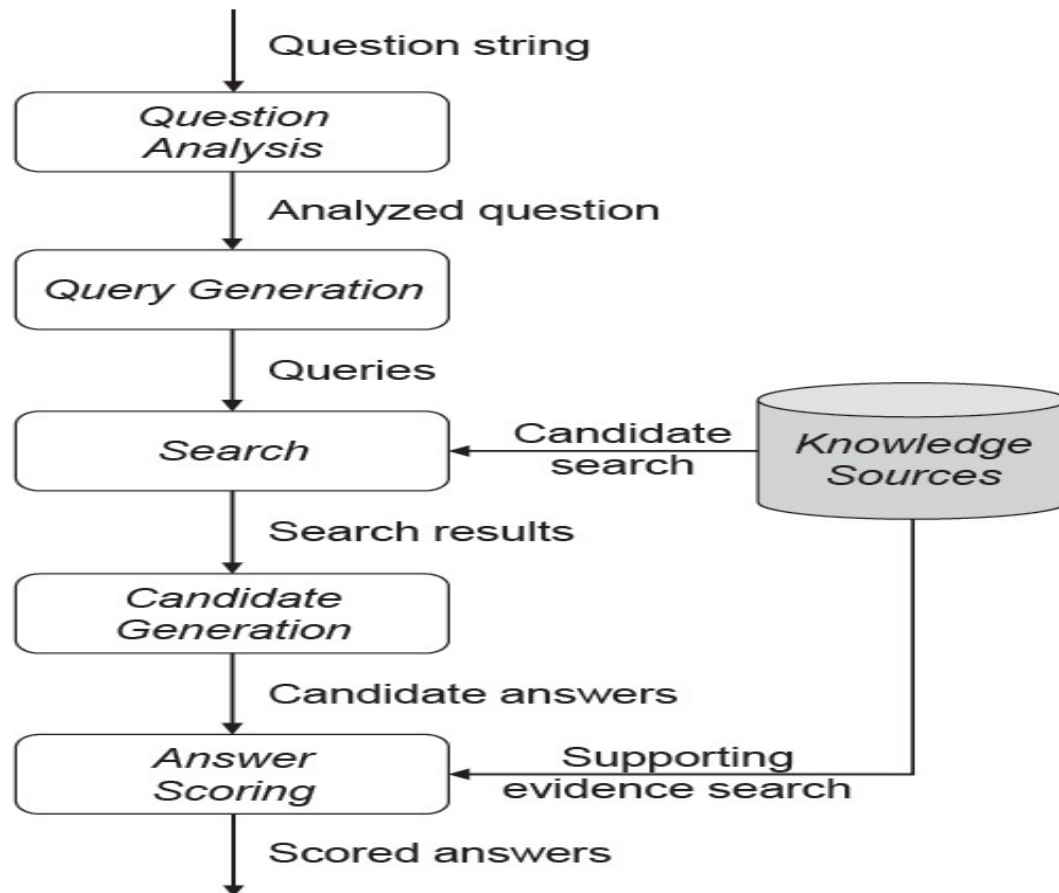
- ❖ Question Answering systems have transformed much in past four decades at par with the whole of natural language processing.
- ❖ It was in the year 1978 when the first classic QA book was published. It was based on Lehnert's thesis where she proposed a question answering system based on semantics and reasoning.
- ❖ We must not be mistaken, question answering systems were at place much before the publication of the book, the story dating back to sixties (Tomek S., Sanda H. , 2010).
- ❖ 1990s saw advancements in the development of question answering systems. We would like to take a look at START, the question answering system developed by Boris Katz.
- ❖ It was first web based question answering system which is continuously running since December 1993.

Question Answering

Architectures:

- ❖ The **question analysis** component derives syntactic and semantic information from the question, using techniques such as answer type classification, syntactic and semantic parsing, and named entity recognition.
- ❖ In the **query generation** stage, this information is transformed into a set of search queries, often with some degree of query expansion, which are passed to the **search** component to retrieve relevant content from a collection of **knowledge sources**.
- ❖ The search results are processed by the **candidate generation** component, which extracts or generates candidate answers of the desired granularity (e.g., factoid answers or definitional phrases).
- ❖ The **answer scoring** component estimates confidence scores for the candidate answers and often merges similar candidates.
- ❖ At this stage, the knowledge sources can be reused to retrieve supporting evidence for individual candidates. The final output is a list of answers ranked by confidence estimates.

Question Answering : Architecture



“Which computer scientist invented the smiley?”



Answer type: computer scientist
Keywords: invented, smiley



“computer scientist” invented smiley



The two original text smileys were invented on **September 19, 1982** by **Scott E. Fahlman** at **Carnegie Mellon**.



- September 19, 1982
- Scott E. Fahlman
- Carnegie Mellon



Answer	Score
Scott E. Fahlman	0.9
Carnegie Mellon	0.4
September 19, 1982	0.3

Question Answering

Source Acquisition and Preprocessing:

- ❖ Source acquisition and preprocessing are essential steps in developing a question answering (QA) system.
- ❖ These steps involve collecting and preparing the data sources that the QA system will use to generate answers to user queries.

Data Sources Selection: The first step is to identify the data sources that the QA system will use. These sources can vary widely, depending on the domain and the specific type of questions the system is designed to answer. Common sources include text documents, databases, websites, and other structured or unstructured data.

Data Collection: Once the data sources are selected, the next step is to collect the relevant data. This may involve web scraping, API calls, document retrieval, or any other method suitable for accessing the chosen sources.

Question Answering

Source Acquisition and Preprocessing:

❖ **Data Corpus Creation:** The collected data is then compiled into a corpus. This corpus is a structured collection of documents or data that the QA system will search for answers. It may involve cleaning the data, removing duplicates, and ensuring that it is in a format suitable for processing.

❖ **Data Annotation:** Depending on the type of QA system, data may need to be annotated. Annotation involves marking up the data with information such as entity recognition, relationships between entities, and any other information that can aid in finding answers. Annotation can be done manually or through natural language processing techniques.

❖ **Text Preprocessing:**

❖ **Text Cleaning**

❖ **Tokenization:**

❖ **Stop Word Removal:**

❖ **Sentence Segmentation:**

❖ **Embedding:**

Question Answering : Question Analysis



Learning from Experience, Sharing with Passion

Question Analysis:

Question analysis is a critical component of question answering (QA) systems. It involves understanding and dissecting user queries to determine their intent, structure, and key components. Effective question analysis helps QA systems identify the type of question being asked and extract the information needed to provide relevant answers.

Question Type Identification:

One of the first tasks in question analysis is identifying the question type. This classification helps determine the method and resources needed to answer the question. Common question types include:

- Fact-based questions (e.g., "What is the capital of France?").
- Descriptive questions (e.g., "Can you explain how photosynthesis works?").
- Comparative questions (e.g., "What are the differences between DNA and RNA?").
- Yes/no questions (e.g., "Is the sky blue?").
- Opinion-based questions (e.g., "What is the best programming language?").

Question Answering

Question Analysis:

Entity Recognition:

Identifying named entities, such as people, places, organizations, and dates, is crucial for understanding the question and finding relevant answers. Named entity recognition (NER) techniques can be used for this purpose.

Key Information Extraction:

Determining the key information in the question is essential for identifying the subject, context, or parameters needed to find an answer. This may involve recognizing verbs, adjectives, or other relevant terms.

Temporal and Spatial Context:

Understanding the temporal and spatial context of a question is important, as it can affect the interpretation of the question and the selection of relevant data sources. For example, "Where was the Eiffel Tower built?" requires knowledge of both the location and the historical context.

Question Answering

Question Analysis:

Coreference Resolution:

Resolving pronouns and other expressions that refer to previously mentioned entities is crucial for maintaining context and clarity. For instance, if a question contains "it" or "he," the system needs to determine what those pronouns refer to.

Query Reformulation:

In some cases, the original user query may need to be reformulated to be more precise or to match the structure of the data sources. This may involve adding context, expanding abbreviations, or refining the language used in the question.

Contextual Analysis:

Understanding the broader context in which the question is asked is important. It may involve analyzing previous questions, user history, or the current conversation to provide more relevant answers.

Search and Candidate Extraction:

Search and candidate extraction are crucial steps in the question answering (QA) process. These steps involve finding and selecting relevant information from a set of data sources to generate answers to user queries.

Search:

- a. **Data Retrieval:** The first step in search is to retrieve relevant data sources that may contain answers to the user's question. These sources can include text documents, databases, websites, or any structured or unstructured data repositories.
- b. **Query Formulation:** The user's question is transformed into a query suitable for searching the data sources. This query may need to be structured to match the search capabilities of the data sources, such as using Boolean operators, wildcards, or other search syntax.

Question Answering

Search:

c. **Indexing:** To expedite the search process, an index of the data sources may be created. This index helps in quickly identifying relevant documents or passages without scanning the entire corpus during each search.

d. **Information Retrieval Techniques:** Various information retrieval techniques, such as keyword matching, vector space models, or advanced machine learning models, can be employed to rank and retrieve documents that are likely to contain relevant information.

Question Answering

Candidate Extraction:

- a. **Document Retrieval:** After the search, a set of candidate documents or passages is retrieved based on their relevance to the query. These candidates are potential sources of answers.
- b. **Ranking:** The retrieved candidates are often ranked by their relevance to the query. Machine learning models or relevance scores generated during the search phase can help determine the order of candidates.
- c. **Passage Selection:** In QA systems, answers are often found within specific passages or sentences. Passage selection involves choosing the most relevant passages from the candidate documents based on their potential to contain answers.
- d. **Answer Type Identification:** The QA system identifies the expected type of answer based on the question. For instance, if the question is asking for a date, location, or person, the system should identify the type and filter out irrelevant passages.

Question Answering

Candidate Extraction:

- e. **Entity Recognition:** In some cases, the QA system may need to recognize specific named entities within the candidate passages. This is crucial for understanding the context and extracting accurate answers.
- f. **Text Preprocessing:** The text within candidate passages may undergo further preprocessing, which can include sentence segmentation, tokenization, and entity recognition to prepare the text for answer extraction.

Question Answering

Candidate extraction from unstructured sources :

- Candidate extraction from unstructured sources refers to the process of identifying and selecting potential candidates or entities of interest from a collection of unstructured data.
- Unstructured sources typically include text documents, such as articles, web pages, social media posts, emails, and more.
- The goal is to extract relevant information and convert it into a structured format for further analysis or use in various applications, such as information retrieval, data mining, natural language processing, or knowledge management.

Text Collection:

Gather a corpus of unstructured text data from various sources relevant to your task.

Preprocessing:

Text preprocessing is essential to clean and prepare the data for analysis. This may involve tasks such as tokenization, lowercasing, removing punctuation, and handling special characters.

Question Answering



Learning from Experience, Sharing with Passion

Candidate extraction from unstructured sources :

Candidate Entity Recognition:

Identify the entities or candidates you want to extract from the text. These entities can be names of people, organizations, locations, dates, products, or any other specific information relevant to your use case.

Named Entity Recognition (NER):

NER is a common technique for identifying entities in text. It involves classifying words or phrases in text into predefined categories, such as "person," "organization," "location," "date," etc. Various NER models and libraries are available, like spaCy, NLTK, or Stanford NER.

Pattern Matching:

Use regular expressions or custom patterns to find specific patterns or formats that represent your candidates. For example, you might use regular expressions to find email addresses, phone numbers, or custom identifiers in text.

Question Answering



Learning from Experience, Sharing with Passion

Candidate extraction from unstructured sources :

Machine Learning:

Train a machine learning model to identify candidates. You can use supervised learning techniques and annotated data to build a model capable of extracting specific entities or information from text.

Rule-Based Systems:

Develop rule-based systems that apply domain-specific or context-specific rules to identify candidates. These rules may be based on keywords, linguistic patterns, or contextual clues.

Post-processing:

After candidate extraction, you may need to perform additional processing steps, such as disambiguation (resolving multiple entities with the same name), normalization (standardizing entity formats), or filtering (removing irrelevant candidates).

Question Answering



Learning from Experience, Sharing with Passion

Candidate extraction from unstructured sources :

Evaluation and Validation:

Assess the performance of your candidate extraction system by comparing the extracted candidates with a ground truth or a gold standard dataset. Evaluate precision, recall, and F1-score to measure the system's accuracy.

Iteration:

Improve your candidate extraction system iteratively by refining rules, enhancing machine learning models, or adjusting parameters based on performance feedback.

Question Answering



Learning from Experience, Sharing with Passion

Candidate extraction from unstructured sources :

Evaluation and Validation:

Assess the performance of your candidate extraction system by comparing the extracted candidates with a ground truth or a gold standard dataset. Evaluate precision, recall, and F1-score to measure the system's accuracy.

Iteration:

Improve your candidate extraction system iteratively by refining rules, enhancing machine learning models, or adjusting parameters based on performance feedback.

Type-Based Candidate Extraction:

❖ Type-based candidate extraction is a process used in information retrieval and natural language processing to identify and extract relevant entities or candidates from a given text based on their semantic types or categories.

❖ This technique is often used in various applications such as named entity recognition (NER), information retrieval, question answering, and more.

Question Answering

Candidate Extraction Using Structural Matching

❖ Candidate extraction using structural matching is a technique employed in natural language processing (NLP) and information retrieval to identify and extract candidates from a given text based on their structural patterns or templates.

❖ This approach involves looking for specific syntactic or structural patterns within the text that correspond to the desired candidates.

❖ **Structural Patterns or Templates:**

First, you define the structural patterns or templates that represent the desired candidates. These patterns can be specified using regular expressions, syntax trees, or other language-specific structures. The templates should capture the syntax and structure of the candidates you want to extract.

Question Answering



Learning from Experience, Sharing with Passion

Candidate Extraction Using Structural Matching

Text Preprocessing: Similar to type-based candidate extraction, you may need to preprocess the text. This can include tokenization, parsing, and syntactic analysis to break down the text into its linguistic components.

Structural Matching: Apply the defined structural patterns or templates to the preprocessed text. The matching process involves looking for instances where the text conforms to the specified structural patterns. When a match is found, it suggests the presence of a candidate.

Candidate Extraction: Extract candidates based on the identified matches. The extracted candidates should conform to the structural patterns specified in the templates. These candidates can be individual words, phrases, or more complex structures, depending on the requirements of your application.

Post-processing: Additional post-processing steps may be necessary to refine the extracted candidates. This can include removing duplicates, filtering out irrelevant candidates, or resolving ambiguities.

Question Answering

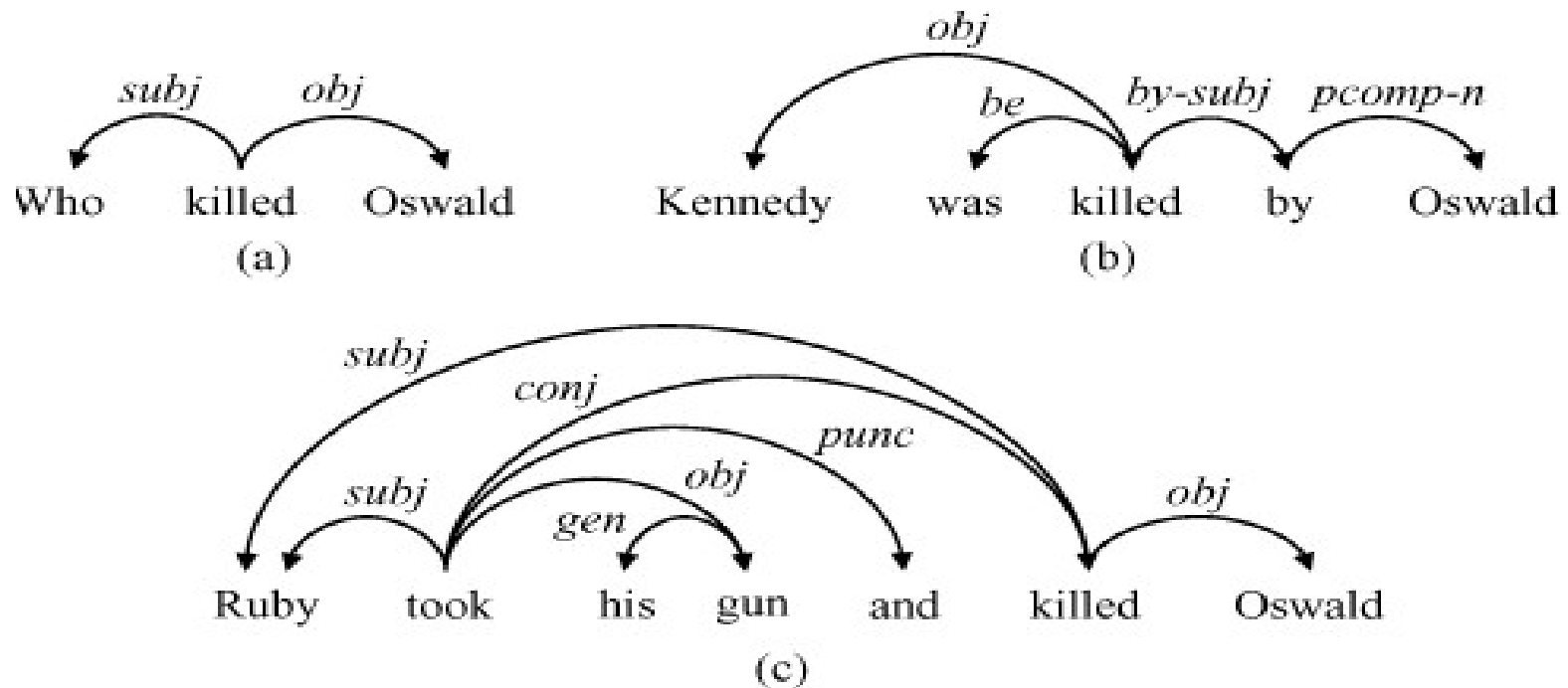
Candidate Extraction Using Structural Matching

Candidate extraction using structural matching is commonly used in various NLP applications, including:

- **Information Retrieval:** Finding documents or passages that contain specific structural patterns, such as legal citations, email addresses, or code snippets.
- **Keyphrase Extraction:** Identifying key phrases or terms in a document that follow particular syntactic patterns, such as noun phrases or verb phrases.
- **Named Entity Recognition:** Recognizing named entities like dates, addresses, or product codes based on their structural characteristics.
- **Text Classification:** Categorizing text based on the presence of specific structural patterns, which can help with tasks like sentiment analysis or topic modeling.
- **Question Answering:** Extracting relevant portions of text that match the syntactic structure of a user's query.

Question Answering

Candidate Extraction Using Structural Matching



Dependency parse trees for a question (a), and two candidate sentences (b, c)

Question Answering

Surface Patterns for Candidate Extraction

- Candidate answers can also be extracted with surface patterns that are solely based on the lexical representation of the search results and do not require a syntactic or semantic analysis.
- The patterns can take the form of regular expression templates that are instantiated with question terms and match text passages that contain the answer.

For instance, the template

< ORG > *was (founded / established) in(the year)?* <ANSWER>.

- may be used to extract candidate answers for a question about the *founding date* of a given organization.
- Questions are categorized by a classification component, and appropriate surface patterns are selected on the basis of the assigned category.
- The categories are not necessarily identical to answer types but can be of a coarser or finer granularity.

Question Answering

Surface Patterns for Candidate Extraction

- Surface patterns for candidate extraction refer to the use of simple, surface-level patterns or regular expressions to identify and extract specific candidates or information from text.
- These patterns are typically based on the way the text is formatted, the arrangement of characters, and other shallow structural characteristics.
- Surface patterns are often used in information extraction tasks when more sophisticated techniques like semantic analysis or deep parsing are not necessary.

For instance, the template

< ORG > was (*founded / established*) in(*the year*)? <ANSWER>.

- may be used to extract candidate answers for a question about the *founding date* of a given organization.
- Questions are categorized by a classification component, and appropriate surface patterns are selected on the basis of the assigned category.
- The categories are not necessarily identical to answer types but can be of a coarser or finer granularity.

Question Answering



Learning from Experience, Sharing with Passion

Surface Patterns for Candidate Extraction

Date Patterns:

Date formats like "MM/DD/YYYY" or "DD-MON-YYYY" can be used to extract dates from text.

Regular expressions can capture dates in various formats, such as "\d{1,2}/\d{1,2}/\d{4}" to extract dates in "MM/DD/YYYY" format.

Email Address Patterns:

To extract email addresses, you can use a pattern like "[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+.[a-zA-Z]{2,}" to match common email address structures.

Phone Number Patterns:

Surface patterns can be designed to capture phone numbers in different formats, like "(###) ###-####" or "###-###-####."

Question Answering



Learning from Experience, Sharing with Passion

Surface Patterns for Candidate Extraction

Web URL Patterns:

A regular expression like "https?:/\S+" can help identify web URLs within text.

Hashtags and Mentions:

For social media data, patterns like "#\w+" for hashtags or "@\w+" for mentions can be used to extract relevant content.

Numeric Patterns:

To extract numerical values, you can use patterns like "\d+" to capture integers or "\d+\.\d+" to capture decimal numbers.

Currency Patterns:

Surface patterns can identify currency amounts, such as "\$100" or "€50," using regular expressions like "[\$€£]\d+".

Question Answering



Learning from Experience, Sharing with Passion

Surface Patterns for Candidate Extraction

Acronyms and Abbreviations:

Surface patterns can be used to extract acronyms or abbreviations that follow certain formatting rules, such as "U.S." for the United States or "NLP" for natural language processing.

Product Codes or Serial Numbers:

If product codes or serial numbers follow specific patterns, regular expressions can be used to capture them, such as "ABC-\d{4}-\w{2}".

Basic Named Entities:

Simple patterns can be used to capture named entities like capitalized words following specific capitalization rules, such as "([A-Z][a-z]+)+[A-Z][a-z]+" for person names.

Question Answering

Candidate Extraction from Structured Sources



Candidate extraction from structured sources typically refers to the process of identifying and selecting relevant items, entities, or records from structured data sources, such as databases, spreadsheets, or tables.

➤ This is a common task in data analysis, data mining, and information retrieval, and it's often a crucial step in various applications, including information retrieval, recommendation systems, and data analytics.

➤ **Data Source Selection:** The first step is to identify and access the structured data sources that contain the information you want to extract candidates from. This can include databases, data warehouses, Excel spreadsheets, CSV files, or other structured data formats.

➤ **Schema Understanding:** It's essential to understand the schema or structure of the data source. This includes knowing the names and types of columns or fields, as well as any relationships between different tables or data sources.

Question Answering



Learning from Experience, Sharing with Passion

Candidate Extraction from Structured Sources

- **Querying and Filtering:** Depending on your specific task, you may need to write SQL queries or use other data manipulation techniques to filter and select the data that is relevant to your candidates. This often involves specifying conditions and criteria to narrow down the dataset.
- **Normalization and Data Cleaning:** It's common for structured data to have inconsistencies, missing values, or errors. Preprocess the data by cleaning and normalizing it to ensure that the extracted candidates are accurate and consistent.
- **Feature Engineering:** Depending on your use case, you may need to create new features or attributes that will be used for candidate extraction. This can involve combining, transforming, or aggregating existing data fields.
- **Candidate Selection Criteria:** Define the criteria that will be used to determine whether a particular item or record qualifies as a candidate for your analysis. These criteria can be based on specific values, patterns, or statistical measures.

Question Answering



Learning from Experience, Sharing with Passion

Candidate Extraction from Structured Sources

- **Algorithmic Selection:** Use algorithms or methods to automatically identify and extract candidates based on the defined selection criteria. This can include simple filtering, regular expressions, clustering, classification, or machine learning techniques, depending on the complexity of the task.
- **Manual Verification:** In some cases, human verification may be necessary to ensure the quality of the extracted candidates, especially when dealing with complex or ambiguous data.
- **Scalability and Performance:** Consider the scalability and performance of your candidate extraction process, especially if you're working with large datasets. Efficient database queries and data processing techniques are essential.
- **Iteration and Refinement:** Candidate extraction is often an iterative process. You may need to adjust your criteria, algorithms, or data preprocessing steps based on the quality of the extracted candidates and the feedback from downstream tasks.
- **Documentation and Reporting:** Maintain documentation of the candidate extraction process, including the selection criteria and any manual verification steps, to ensure transparency and reproducibility.



Lecture-#20

Natural Language Processing

A8706

UNIT-V

Text Summarization and Information Retrieval

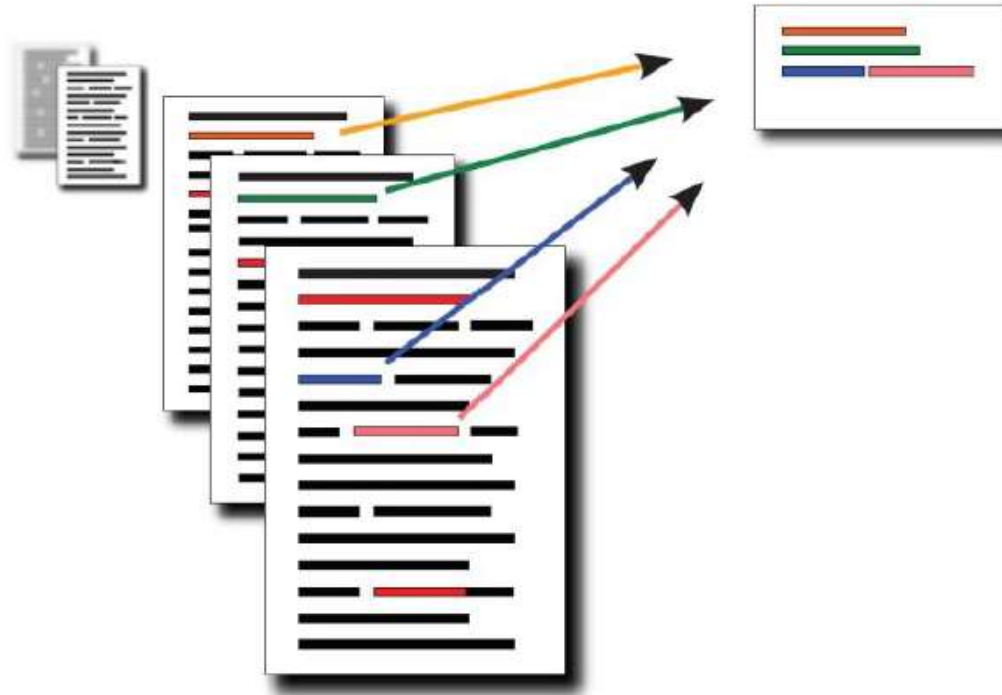
Text Summarization:

- ❖ Text summarization refers to the technique of shortening long pieces of text.
- ❖ The intention is to create a coherent and fluent summary having only the main points outlined in the document.
- ❖ Automatic text summarization is a common problem in machine learning and natural language processing (NLP).
- ❖ Text summarization is the process of creating a shorter version of the text with only vital information and thus, helps the user to understand the text in a shorter amount of time.
- ❖ The main advantage of text summarization lies in the fact that it reduces user's time in searching the important details in the document.

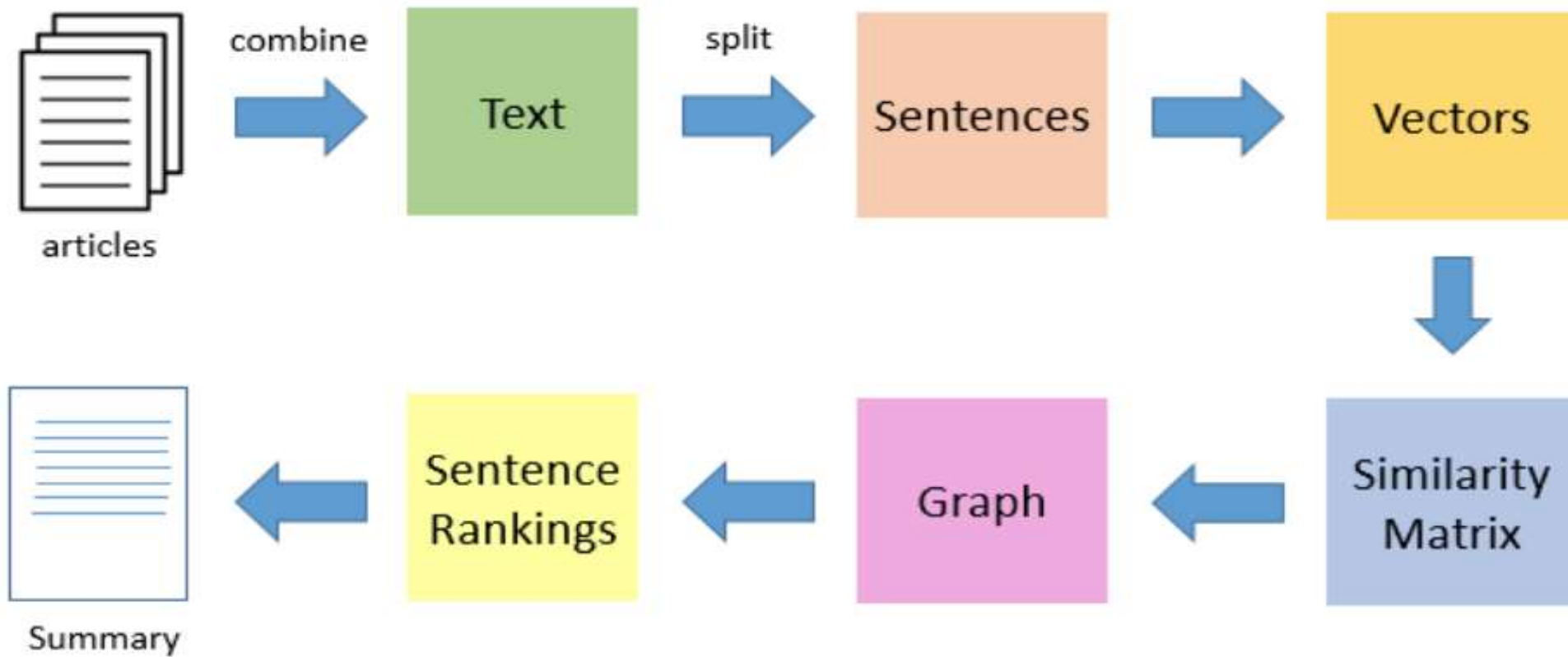
Text Summarization

What is it?

Task: produce an abridged version of a text while retaining the key, relevant information



Text Summarization



Approaches to Summarization

Single-document summarization

Given a single doc, produce:

- Abstract
- Outline
- Headline

Multiple-document summarization

Given a group of docs, produce:

- Series of news stories on the same event
- Set of web pages about some topic or question

Approaches to Summarization

Single-doc summarization

Document

Cambodian leader Hun Sen on Friday rejected opposition parties ' demands for talks outside the country , accusing them of trying to " internationalize " the political crisis .
Government and opposition parties have asked King Norodom Sihanouk to host a summit meeting after a series of post-election negotiations between the two opposition groups and Hun Sen 's party to form a new government failed .
Opposition leaders Prince Norodom Ranariddh and Sam Rainsy , citing Hun Sen 's threats to arrest opposition figures after two alleged attempts on his life , said they could not negotiate freely in Cambodia and called for talks at Sihanouk 's residence in Beijing .Hun Sen , however , rejected that .''
I would like to make it clear that all meetings related to Cambodian affairs must be conducted in the Kingdom of Cambodia , " Hun Sen told reporters after a Cabinet meeting on Friday ,'' No-one should internationalize Cambodian affairs .
It is detrimental to the sovereignty of Cambodia , " he said .Hun Sen 's Cambodian People 's Party won 64 of the 122 parliamentary seats in July 's elections , short of the two-thirds majority needed to form a government on its own .Ranariddh and Sam Rainsy have charged that Hun Sen 's victory in the elections was achieved through widespread fraud .They have demanded a thorough investigation into their election complaints as a precondition for their cooperation in getting the national assembly moving and a new government formed



Summary

Cambodian government rejects
opposition's call for talks abroad

Approaches to Summarization

Multi-doc summarization

Documents

Fingerprints and photos of two men who boarded the doomed Malaysia Airlines passenger jet are being sent to U.S. authorities so they can be compared against records of known terrorists and criminals. The cause of the plane's disappearance has baffled investigators and they have not said that they believed that terrorism was involved, but they are also not ruling anything out. The investigation into the disappearance of the jetliner with 239 passengers and crew has centered so far around the fact that two passengers used passports stolen in Thailand from an Austrian and an Italian. The plane which left Kuala Lumpur, Malaysia, was headed for Beijing. Three of the passengers, one adult and two children, were American.

(CNN) -- A delegation of painters and calligraphers, a group of Buddhists returning from a religious gathering in Kuala Lumpur, a three-generation family, nine senior travelers and five toddlers. Most of the 227 passengers on board missing Malaysia Airlines Flight 370 were Chinese, according to the airline's flight manifest. The 12 missing crew members on the flight that disappeared early Saturday were Malaysian. The airline's list showed the passengers hailed from 14 countries, but later it was learned that two people named on the manifest -- an Austrian and an Italian -- whose passports had been stolen were not aboard the plane. The plane was carrying five children under 5 years old, the airline said.

⋮

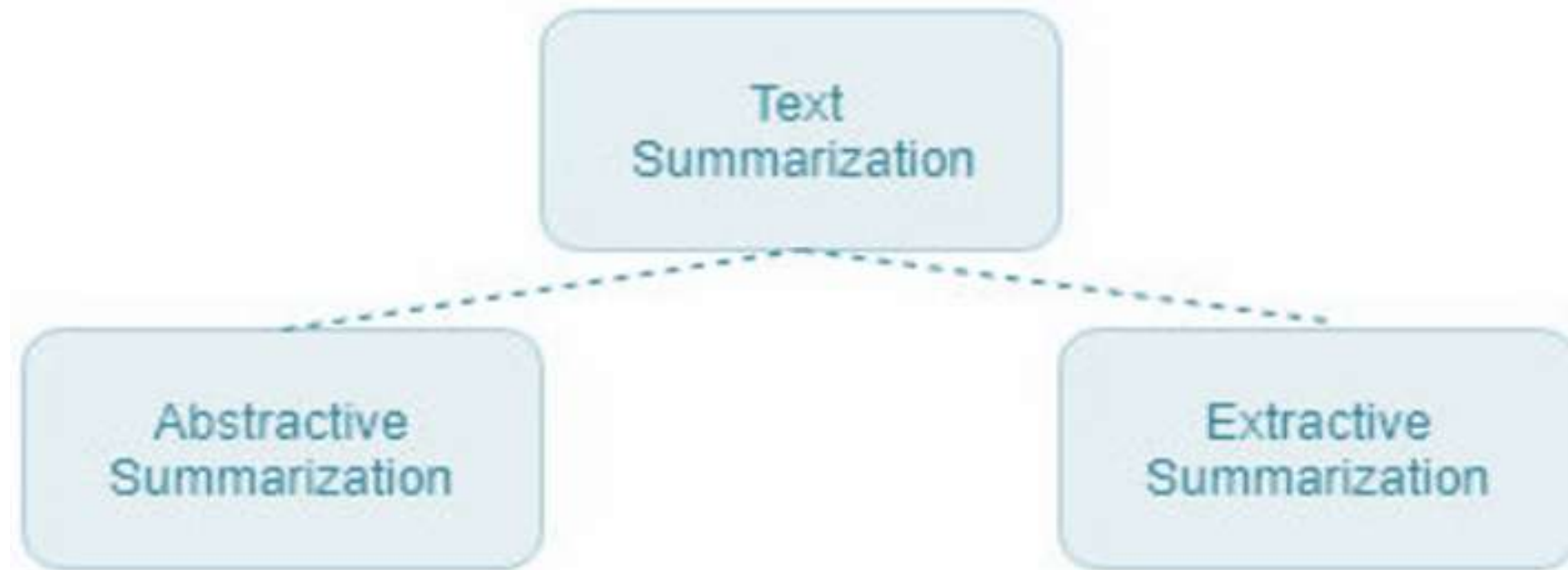
Vietnamese aircraft spotted what they suspected was one of the doors belonging to the ill-fated Malaysia Airlines Flight MH370 on Sunday, as troubling questions emerged about how two passengers managed to board the Boeing 777 using stolen passports. The discovery comes as officials consider the possibility that the plane disintegrated mid-flight, a senior source told Reuters. The state-run Thanh Nien newspaper cited Lt. Gen. Vo Van Tuan, deputy chief of staff of Vietnam's army, as saying searchers in a low-flying plane had spotted an object suspected of being a door from the missing jet. It was found in waters about 56 miles south of Tho Chu island, in the same area where oil slicks were spotted Saturday.

Summary

Flight MH370, carrying 239 people vanished over the South China Sea in less than an hour after taking off from Kuala Lumpur, with two passengers boarded the Boeing 777 using stolen passports. Possible reasons could be an abrupt breakup of the plane or an act of terrorism. The government was determining the "true identities" of the passengers who used the stolen passports. Investigators were trying to determine the path of the plane by analysing civilian and military radar data while ships and aircraft from seven countries scouring the seas around Malaysia and south of Vietnam.

Approaches to Summarization

There are two main approaches to summarizing text documents –



Approaches to Summarization

There are two main approaches to summarizing text documents –

Extractive Method: It involves selecting phrases and sentences from the original text and including it in the final summary.

Example:

Original Text : Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is a great language for the beginner-level programmers.

Extractive Summary : Python is a high-level scripting language is great language for beginner-level programmers.

Approaches to Summarization

Abstractive Method: The Abstractive method involves generating entirely new phrases and sentences to capture the meaning of source document.

Example:

Original Text : Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is a great language for the beginner-level programmers

Abstractive Summary : Python is interpreted and interactive language and it is easy to learn.

Approaches to Summarization

Extractive summarization

Select from the source text spans that capture the key information

Abstractive summarization

Generate new text that encapsulates the key information from the source text

Generic summarization

- Summarize the content of the doc(s)

Query-focused summarization

- Summarize a doc w.r.t. a user's query
- **Complex question-answering** (answer a question by summarizing a doc that has the information to construct the answer)

Approaches to Summarization

Ideal stages of summarization:

❖ Analysis

Input representation and understanding

❖ Transformation

Selecting important content

❖ Realization

Generating novel text corresponding to the gist of the input

Approaches to Summarization

- Extract the key influential phrases from the documents
- Extract various diverse concepts or topics present in the documents
- Summarize the documents to provide a gist that retains the important parts of the whole corpus

Documents

A *document* is usually an entity containing a whole body of text data with optional headers and other metadata information. A corpus usually consists of a collection of documents. These documents can be simple sentences or complete paragraphs of textual information. *Tokenized corpus* refers to a corpus where each document is tokenized or broken down into *tokens*, which are usually words.

Text Normalization

Text normalization is the process of cleaning, normalizing, and standardizing textual data with techniques like removing special symbols and characters, removing extraneous HTML tags, removing stopwords, correcting spellings, stemming, and lemmatization.

Feature Matrix

A *feature matrix* usually refers to a mapping from a collection of documents to features where each row indicates a document and each column indicates a particular feature, usually a word or a set of words. We will represent collections of documents or sentences through feature matrices after feature extraction and we will often apply statistical and ML techniques on these matrices later on in our practical examples.

	[male]	[adult]
man	+	+
woman	-	+
boy	+	-
girl	-	-

Approaches to Summarization

- ❖ Information Extraction (IE) and Text Summarization are two methods of extracting relevant portions of the input text.
- ❖ IE produces templates, whose slots are filled with the important information, while Summarization produces one of various types of summary.
- ❖ Popular Techniques:

keyphrase extraction, topic modeling, and automated document summarization.

Keyword extraction :

- ❖ Keyword extraction is a text analysis approach that extracts the most frequently used and essential words and expressions from a document automatically.
- ❖ It aids in the summarization of text material and the identification of the primary issues presented.

For example, in the text

"The food was delicious and the staff were wonderful.",
key phrase extraction will return the main topics:

?

Keyword extraction :

- ❖ Keyword extraction is a text analysis approach that extracts the most frequently used and essential words and expressions from a document automatically.
- ❖ It aids in the summarization of text material and the identification of the primary issues presented.

For example, in the text

"The food was delicious and the staff were wonderful.",
key phrase extraction will return the main topics:

"food" and "wonderful staff".

Topic modelling:

- ❖ **Topic modelling** is recognizing the words from the topics present in the document or the corpus of data.
- ❖ This is useful because extracting the words from a document takes more time and is much more complex than extracting them from topics present in the document.
- ❖ Topics can be defined as “a repeating pattern of co-occurring terms in a corpus”. A good topic model should result in –

“health”, “doctor”, “patient”, “hospital” for a topic – ?

and

“farm”, “crops”, “wheat” for a topic – ?

Topic modelling:

- ❖ **Topic modelling** is recognizing the words from the topics present in the document or the corpus of data.
- ❖ This is useful because extracting the words from a document takes more time and is much more complex than extracting them from topics present in the document.
- ❖ Topics can be defined as “a repeating pattern of co-occurring terms in a corpus”. A good topic model should result in –
“health”, “doctor”, “patient”, “hospital” for a topic – **Healthcare**, and
“farm”, “crops”, “wheat” for a topic – **Farming**”.

Approaches to Summarization

Automated document summarization:

- ❖ Automated document summarization refers to performing the summarization of a document or documents using some form of heuristics or statistical methods.
- ❖ A summary in this case is a shortened piece of text which accurately captures and conveys the most important and relevant information contained in the document or documents we want summarized.

Applications

These are some use cases where automatic summarization can be used across the enterprise:

1. Media monitoring

The problem of information overload and “[content shock](#)” has been widely discussed. Automatic summarization presents an opportunity to condense the continuous torrent of information into smaller pieces of information.

2. Newsletters

Many weekly newsletters take the form of an introduction followed by a curated selection of relevant articles. Summarization would allow organizations to further enrich newsletters with a stream of summaries (versus a list of links), which can be a particularly convenient format in mobile.

Applications

3. Search marketing and SEO

When evaluating search queries for SEO, it is critical to have a well-rounded understanding of what your competitors are talking about in their content. This has become particularly important since Google updated its algorithm and shifted focus towards [topical authority](#) (versus keywords). Multi-document summarization can be a powerful tool to quickly analyze dozens of search results, understand shared themes and skim the most important points.

4. Internal document workflow

Large companies are constantly producing internal knowledge, which frequently gets stored and under-used in databases as unstructured data. These companies should embrace tools that let them re-use already existing knowledge. Summarization can enable analysts to quickly understand everything the company has already done in a given subject, and quickly assemble reports that incorporate different points of view.

Applications

5. Financial research

Investment banking firms spend large amounts of money acquiring information to drive their decision-making, including automated stock trading. When you are a financial analyst looking at market reports and news everyday, you will inevitably hit a wall and won't be able to read everything. Summarization systems tailored to financial documents like earning reports and financial news can help analysts quickly derive market signals from content.

6. Legal contract analysis

Related to point 4 (internal document workflow), more specific summarization systems could be developed to analyze legal documents. In this case, a summarizer might add value by condensing a contract to the riskier clauses, or help you compare agreements.

Applications

7. Social media marketing

Companies producing long-form content, like whitepapers, e-books and blogs, might be able to leverage summarization to break down this content and make it sharable on social media sites like Twitter or Facebook. This would allow companies to further re-use existing content.

8. Question answering and bots

Personal assistants are taking over the workplace and the smart home. However, most assistants are fairly limited to very specific tasks. Large-scale summarization could become a powerful question answering technique. By collecting the most relevant documents for a particular question, a summarizer could assemble a cohesive answer in the form of a multi-document summary.

9. Video scripting

Video is becoming one of the most important marketing mediums. Besides video-focused platforms like YouTube or Vimeo, people are now sharing videos on professional networks like LinkedIn. Depending on the type of video, more or less scripting might be required. Summarization can get to be an ally when looking to produce a script that incorporates research from many sources.

10. Medical cases

With the growth of tele-health, there is a growing need to better manage medical cases, which are now fully digital. As telemedicine networks promise a more accessible and open healthcare system, technology has to make the process scalable. Summarization can be a crucial component in the tele-health supply chain when it comes to analyzing medical cases and routing these to the appropriate health professional.

Applications

11. Books and literature

Google has reportedly worked on projects that attempt to [understand novels](#). Summarization can help consumers quickly understand what a book is about as part of their buying process.

12. Email overload

Companies like Slack were born to keep us away from constant emailing. Summarization could surface the most important content within email and let us skim emails faster.

13. E-learning and class assignments

Many teachers utilize case studies and news to frame their lectures. Summarization can help teachers more quickly update their content by producing summarized reports on their subject of interest.

Applications

14. Science and R&D

Academic papers typically include a human-made abstract that acts as a summary. However, when you are tasked with monitoring trends and innovation in a given sector, it can become overwhelming to read every abstract. Systems that can group papers and further compress abstracts can become useful for this task.

15. Patent research

Researching patents can be a tedious process. Whether you are doing market intelligence research or looking to file a new patent, a summarizer to extract the most salient claims across patents could be a time saver.

16. Meetings and video-conferencing

With the growth of tele-working, the ability to capture key ideas and content from conversations is increasingly needed. A system that could turn voice to text and generate summaries from your team meetings would be fantastic.

Applications

17. Help desk and customer support

Knowledge bases have been around for a while, and they are critical for SAAS platforms to provide customer support at scale. Still, users can sometimes feel overwhelmed when browsing help docs. Could multi-document summarization provide key points from across help articles and give the user a well rounded understanding of the issue?

18. Helping disabled people

As voice-to-text technology continues to improve, people with hearing disabilities could benefit from summarization to keep up with content in a more efficient way.

19. Programming languages

There have been multiple attempts to build [AI technology that could write code](#) and build websites by itself. It is a possibility that custom “code summarizers” will emerge to help developers get the big picture out of a new project.

20. Automated content creation

“Will robo-writers replace my job?” That’s what writers are increasingly asking themselves. If artificial intelligence is able to replace any stage of the content creation process, automatic summarization is likely going to play an important role. Related to point 3 (applications in search marketing and SEO), writing a good blog usually goes by summarizing existing sources for a given query. Summarization technology might reach a point where it can compose an entirely original article out of summarizing related search results.



Lecture-#21

Natural Language Processing

A8706

UNIT-V

Text Summarization and Information Retrieval

Approaches to Summarization

surface features Extraction :

- ❖ indicative phrases such as *in summary*
- ❖ distribution of terms
- ❖ overlap with words from headings, titles
- ❖ position of sentence in the text, paragraph, and so on

Graph-Based Approaches :

- ❖ Translate the text into a graph representation with similarity scores as weights for the links between nodes representing the sentences.

rhetoical structure theory (RST)

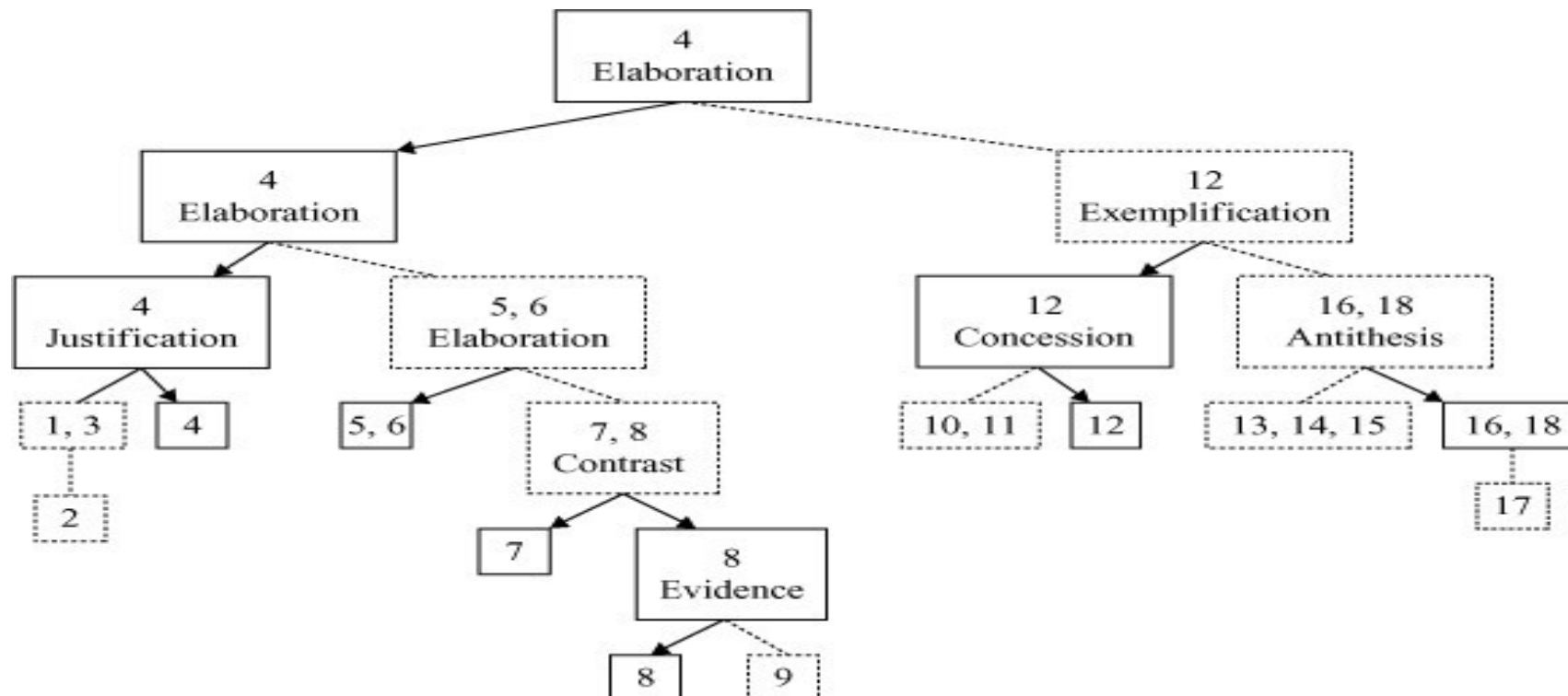
- ❖ The main assumption of RST is based on the observation that text segments are connected via rhetorical relations.
- ❖ A rhetorical relation between text segments can either be explicitly marked via a discourse marker (e.g., *because*) or inferred from the context.
- ❖ Rhetorical relations can express causality between events, elaborate a situation, or move the narration forward.
- ❖ . One of the core ideas of RST is the generation of a discourse tree.
- ❖ The nodes of the tree can combine text spans.

Approaches to Summarization

Figure 12–1 contains an RST discourse tree for the following example text about Mars exploration, as analyzed by Marcu [31]:

[With its distant orbit **1**] [— 50 percent farther from the sun than Earth — **2**] [and slim atmospheric blanket, **3**] [Mars experiences frigid weather conditions. **4**] [Surface temperatures typically average about 60 degrees Celsius (76 degrees Fahrenheit) at the equator **5**] [and can dip to 123 degrees C near the poles. **6**] [Only the midday sun at tropical latitudes is warm enough to thaw ice on occasion, **7**] [but any liquid water formed in this way would evaporate almost instantly **8**] [because of the low atmospheric pressure. **9**] [Although the atmosphere holds a small amount of water, **10**] [and water-ice clouds sometimes develop, **11**] [most Martian weather involves blowing dust or carbon dioxide. **12**] [Each winter, for example, a blizzard of frozen carbon dioxide rages over one pole, **13**] [and a few meters of this dry-ice snow accumulate **14**] [as previously frozen carbon dioxide evaporates from the opposite polar cap. **15**] [Yet even on the summer pole, **16**] [where the sun remains in the sky all day long, **17**] [temperatures never warm enough to melt frozen water.**18**]

Approaches to Summarization



The RST structure of the Mars example text (Reproduced from Marcu]

Text as Graph:

- ❖ TextRank is an approach to summarization that also utilizes a graph representation.
- ❖ TextRank is related to PageRank, but instead of generating a graph consisting of linked documents, the graph is derived from the text.
- ❖ The sentences in the text are represented as the vertices, whereas the edges between the vertices are weighted by a similarity metric.
- ❖ Similar to PageRank, vertices that are highly connected are “recommended” by other sentences and receive a high rank.
- ❖ A graph representation is used for the text where a directed graph $G = (V, E)$ with a set of vertices V and a set of edges $E \subseteq V \times V$ represents links between sentences.
- ❖ The PageRank score is computed on the basis of the number of inlinks $in(V_i)$ pointing to a vertex and the number of outlinks $out(V_i)$ pointing away from a vertex.
- ❖ The PageRank score is then computed via the following formula, where d is the dampening factor indicating the probability of jumping to a new page:

Approaches to Summarization

- The parameter d is normally set to 0.85.

$$S(V_i) = (1 - d) + d * \sum_{j \in in(V_i)} \frac{1}{|out(V_j)|} S(V_j) \quad (12.1)$$

- ❖ For TextRank, the directed graph becomes an undirected graph and consequently $in(V_i) = out(V_i)$.
- ❖ Instead, the links have weights w_{jk} based on similarity scores between the sentences.
- ❖ The similarity metric defined by Mihalcea and Tarau counts the number of words two sentences share and normalizes by the length of the sentences.

$$Similarity(S_i, S_j) = \frac{|\{w_k | w_k \in S_i \wedge w_k \in S_j\}|}{\log(|S_i|) + \log(|S_j|)} \quad (12.2)$$

Approaches to Summarization

The weighted PageRank score is defined as follows:

$$WS(V_i) = (1 - d) + d * \sum_{V_j \in in(V_i)} \frac{1}{\sum_{V_k \in out(V_j)} w_{jk}} WS(V_j) \quad (12.3)$$

Approaches to Summarization

Learning How to Summarize:

- ❖ *An aligned corpus between summaries and their respective document(s).* Some method (e.g., word overlap) has to be employed to match summary sentences and sentences from the text to be summarized.
- ❖ *Feature extractors that generate feature vectors for each sentence.* A feature may be the length of a sentence, the position in the text/paragraph, the overlap of words in the sentence with the title/headings, or the word frequency of the words in the sentence.
- ❖ *A machine learning algorithm that classifies a sentence.* This can be a binary classifier, a multiclass classifier, or a regression model in which a sentence would receive an overall score.

Learning How to Summarize:

- ❖ **Score** Each sentence from the training set is assigned a score.
- ❖ This score may be computed by the word overlap of document sentences with the sentences from model summaries.
- ❖ Given the sentence-score combinations, a regression model can be learned. Support vector regression (SVR) can be employed for learning a score for each sentence.

Approaches to Summarization

- ❖ **Partial order** Pairs of sentences are ranked in order to receive a partial order of sentences.
- ❖ A learning to rank method expects pairs of sentences that learns a partial order.
- ❖ **Ranks** Another way of learning summary sentences is to learn how sentences are ranked in a list. Instead of pairwise ranking, sentences are ranked as a full-order list or at least in several “buckets.”



Lecture-#22

Natural Language Processing

A8706

UNIT-V

Text Summarization and Information Retrieval

Spoken Dialog Systems

- ❖ Spoken dialog systems are probably among the most widely adopted applications of speech recognition.
- ❖ A spoken dialog system is an application whereby a machine communicates with humans using speech in a series of sequential interaction turns.

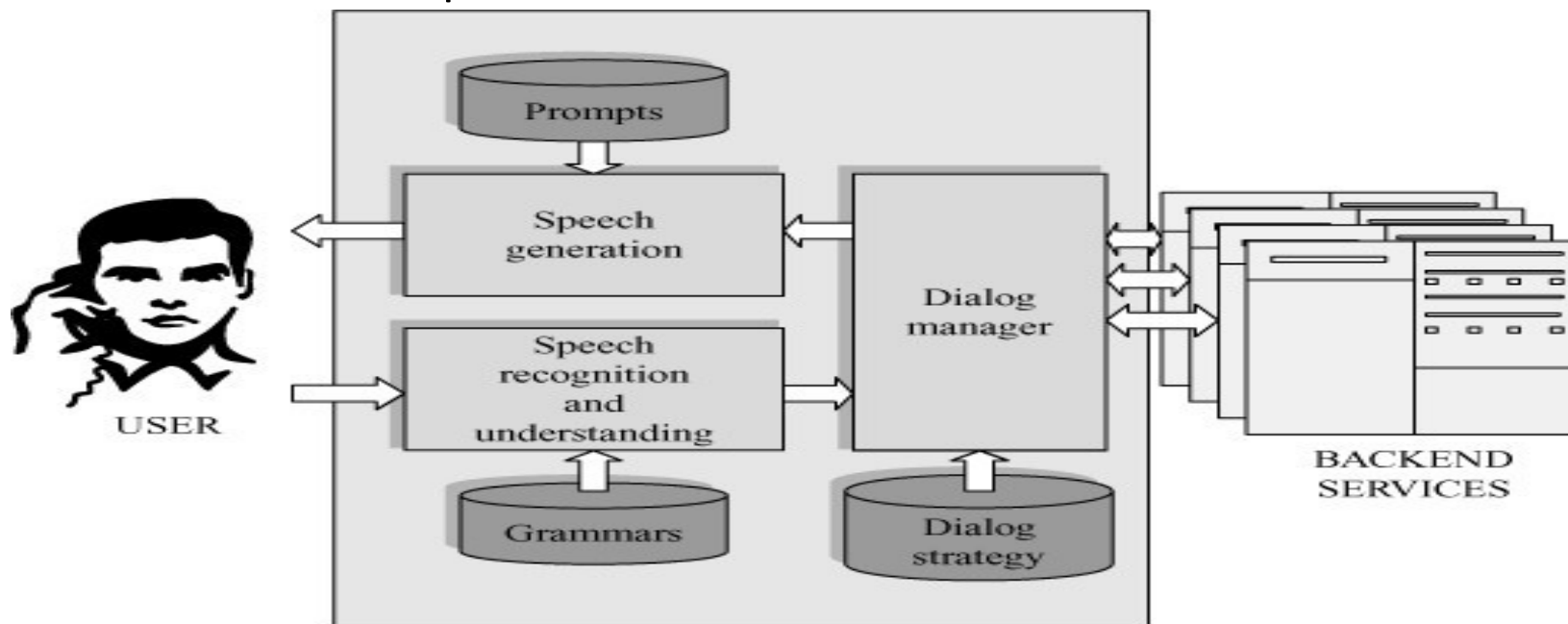


Figure 15–1. A high-level functional diagram of a spoken dialog system

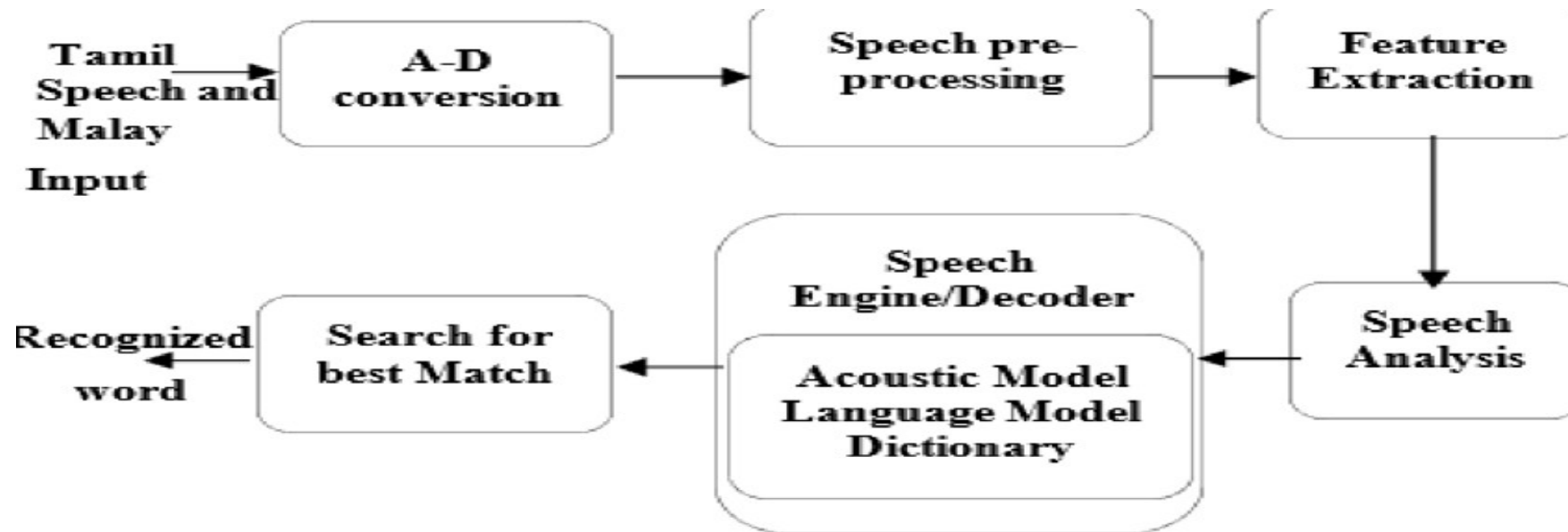
Spoken Dialog Systems

- ❖ The **dialog manager**, based on a set of rules called **dialog strategy**, directs a **speech generation** module on what information or request will be spoken after it has received the interpretation of the user's speech from a **speech recognition and understanding** module.
- ❖ The dialog manager also interacts with external **backend services**, such as databases, customer relationship management (CRM) systems, or the web, to extract additional information necessary to complete the interaction.
- ❖ We should mention that the approaches described here are in principle language independent.
- ❖ However, some languages may require the application of additional processing, mainly at the level of the speech recognition and understanding module, to deal with different conceptual definitions of token.

Spoken Dialog Systems

Speech Recognition and Understanding:

- ❖ A speech recognition engine is a system that decodes input speech into its constituent words.
- ❖ The understanding module is then responsible for assigning a semantic tag to each string of words returned by the speech recognizer.



Spoken Dialog Systems

TRANSCRIPTION	ANNOTATION
want to cancel the account	SERVICE_CANCEL
cancel service	SERVICE_CANCEL
cancellation of the service	SERVICE_CANCEL
I want to discontinue the service	SERVICE_CANCEL
I can't send a particular message to a certain group of people	CANT_SEND_RECEIVE_EMAIL
I can't get messages on my email and Outlook Express	CANT_SEND_RECEIVE_EMAIL
I can't receive all my email	CANT_SEND_RECEIVE_EMAIL
I'm trying to send an email and it says it's not going through	CANT_SEND_RECEIVE_EMAIL
my emails are not being received at the address	CANT_SEND_RECEIVE_EMAIL
I send them to	
can't send	CANT_SEND_RECEIVE_EMAIL
can't send large files	CANT_SEND_RECEIVE_EMAIL
bounce message notification	CANT_SEND_RECEIVE_EMAIL
message won't be sent won't send	CANT_SEND_RECEIVE_EMAIL
it concerns mac mail I can't open it	SETUP_EMAIL
when I set up the internet you didn't give the email account	SETUP_EMAIL
I can't set up my email account	SETUP_EMAIL
setting up email account	SETUP_EMAIL
cannot configure the email	SETUP_EMAIL
they registered my modem from my Internet and	SETUP_EMAIL
I need to get my email address	
all I need is to find out how to set up my sent email box to save my sent email	SETUP_EMAIL
I'd like to set up an additional email account	SETUP_EMAIL

Speech Generation:

- ❖ The speech generation module in commercial spoken dialog systems is quite limited or nonexistent.
- ❖ Research has experimented with natural language generation (NLG) modules followed by text-to-speech (TTS).

Dialog Manager:

- ❖ A spoken dialog system, the dialog strategy is expressed as a **call-flow**.
- ❖ A call-flow corresponds to the specification of a finite state machine, often structured in a hierarchical fashion, with nodes representing dialog activities and arcs corresponding to conditions.

Architecture of a modern commercial spoken dia- log system

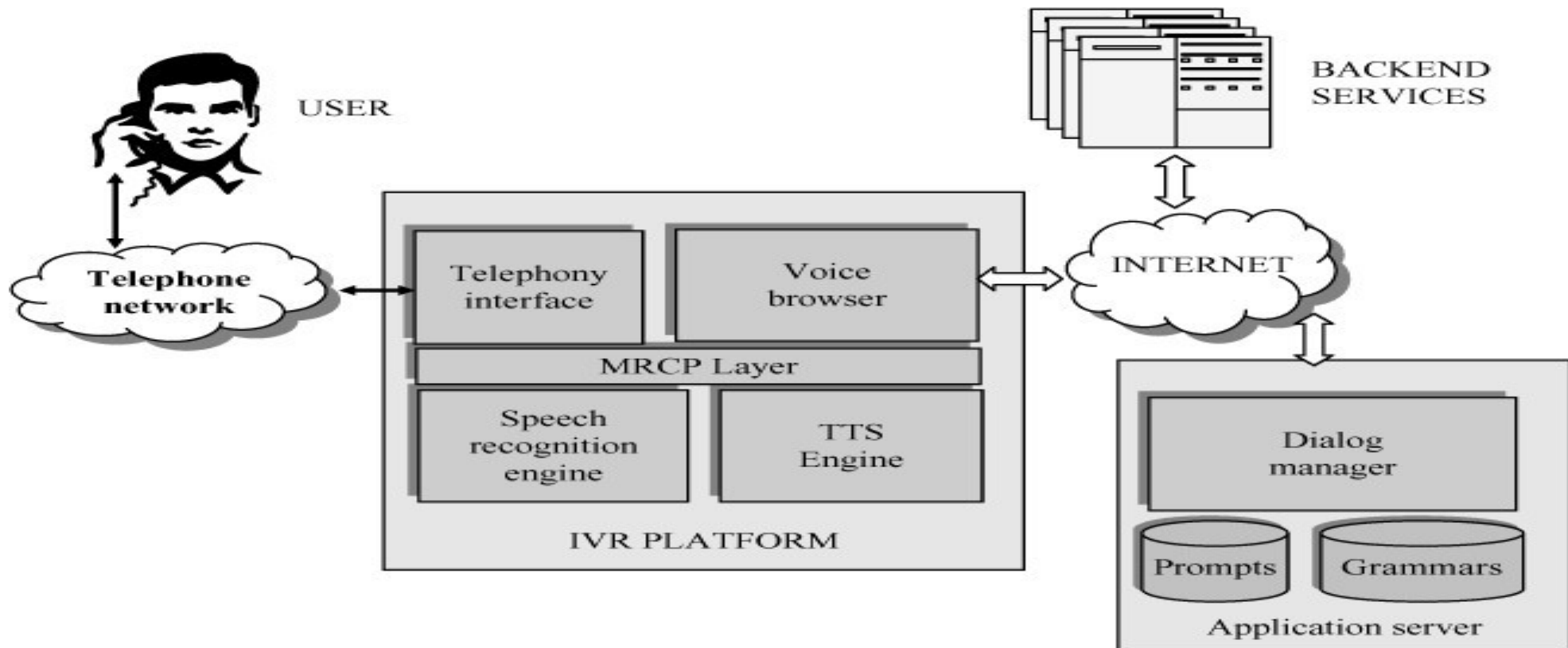


Figure 15–3. Architecture of a modern commercial spoken dialog system

Spoken Dialog Systems

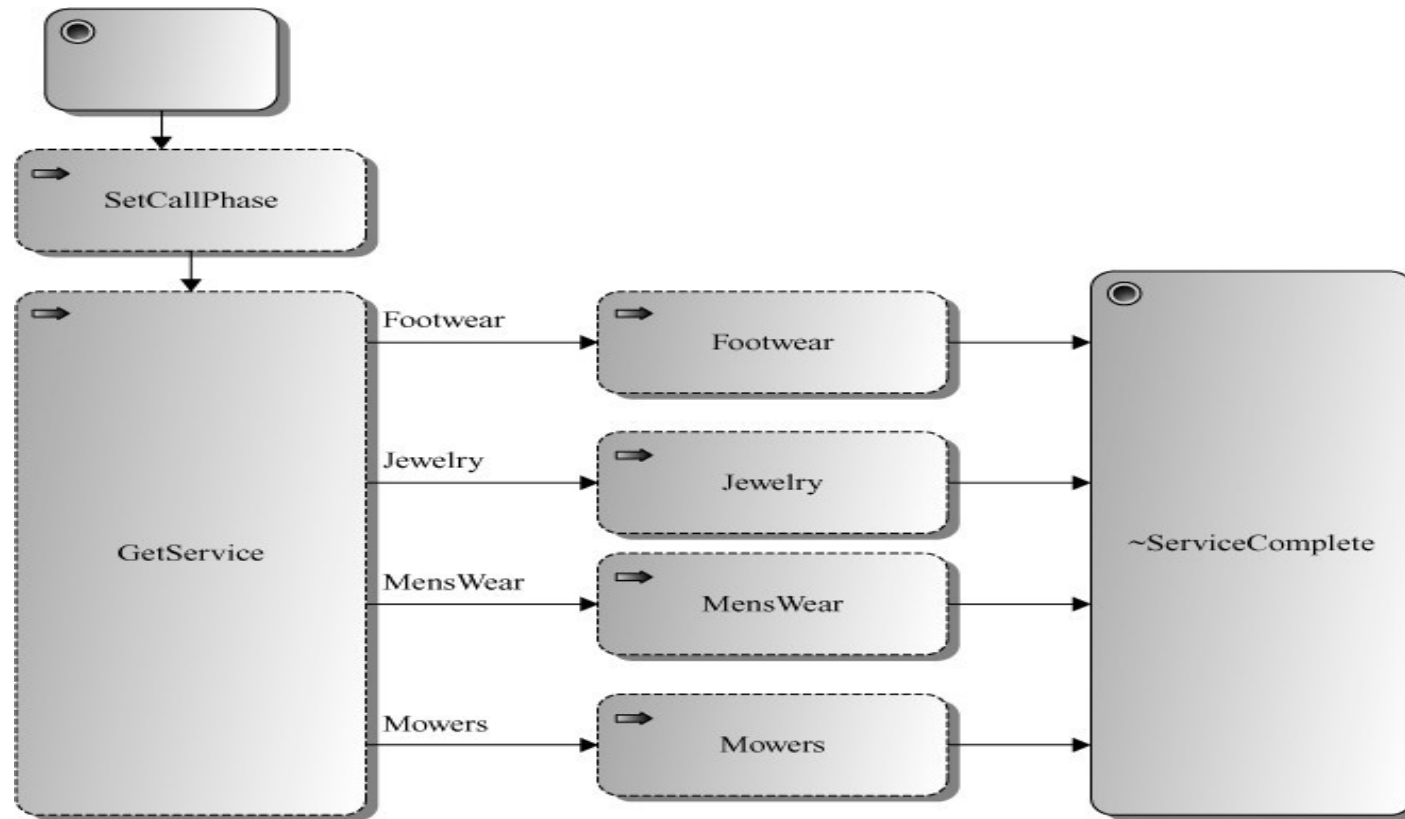
- ❖ The interactive voice response (IVR) platform comprises a VoiceXML browser, which interprets VoiceXML documents, and a telephone interface, which connects to the public telephone network (or equivalently, a Voice over IP gateway).
- ❖ The VoiceXML browser controls standard speech recognition and TTS engines through a Media Resource Control Protocol (MRCP) protocol layer.
- ❖ The interaction is managed by the application server (implemented through a regular Web server), which serves VoiceXML documents upon HTTP requests issued by the IVR platform.
- ❖ Prompts and grammars are generally associated to URLs and may be hosted by the same web server that hosts the application server or elsewhere in the network.
- ❖ The dialog manager may occasionally access backend services, often using SOAP (Simple Object Access Protocol) directives.

Voice User Interface

- ❖ The voice user interface (VUI) specification for a given application is what describes the system's prompts, what is the range of expressions accepted by the speech recognizer at each step of the interaction, and the general logic of the application.
- ❖ A call-flow is a finite-state machine description of an application VUI.
- ❖ VUIs are often designed and developed using drag- and-drop WYSIWYG tools that allow visualizing all the details of the interaction as a hierarchical finite-state machine.
- ❖ Call-flow authoring tools compile the graphical representation in a call-flow markup language— typically a proprietary language—which is then used by a call-flow application engine to render the interaction by generating VoiceXML dynamically.

Spoken Dialog Systems

Voice User Interface



. Example call-flow specification built with a WYSIWYG authoring tool

Voice User Interface

- ❖ Boxes represent activities, which are the equivalent of the statements of a traditional procedural language.
- ❖ Arcs represent conditional transitions to other activities; that is, they are the equivalent of if-then-else clauses of a traditional procedural language.
- ❖ The first activity in the upper left corner of **Figure 15–4** is the entry point of the process, and the activity to the right of the chart, indicated by ~ServiceComplete , indicates the return to the calling process (the Main process in this case).
- ❖ All the other activities in **Figure 15–4** are process references: they correspond to the invocation of subroutines defined by other graphical specifications (or pages) similar to the one of **Figure 15–4**.
- ❖ Process references are the equivalent of function calls in a traditional language. For instance, the GetService process is specified by the graphical page of **Figure 15–5**.

Spoken Dialog Systems

Voice User Interface

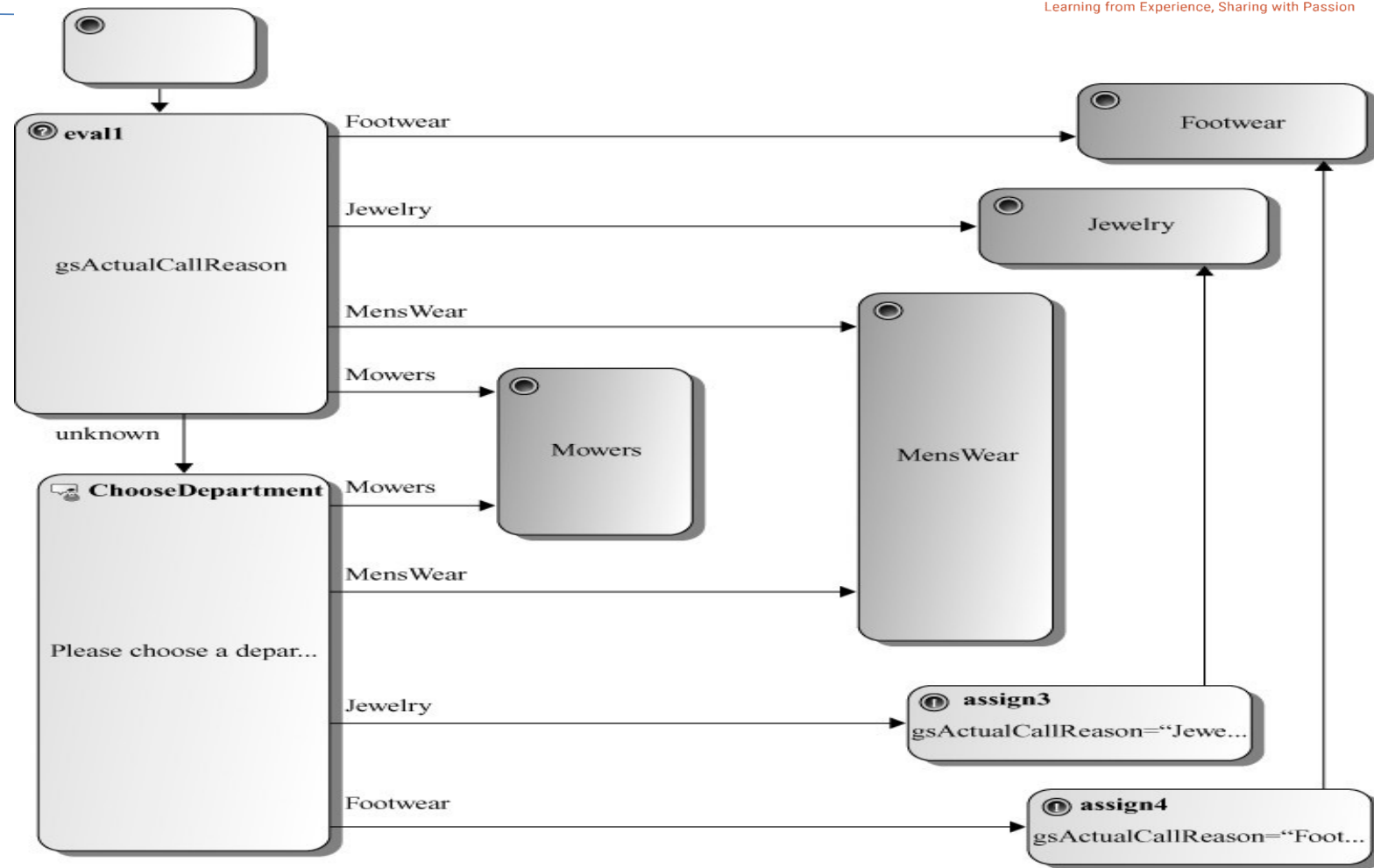


Figure 15–5. Expansion of the GetService process referenced in [Figure 15–4](#)

Voice User Interface

- ❖ The activity called eval1 (upper left of **Figure 15–5**) corresponds to the evaluation of the variable getActualCallReason .
- ❖ If the variable evaluates as one of the four possible values (Footwear, Jewelry, MensWear, Mowers), the current process returns to the calling one with the appropriate return value.
- ❖ If the variable is unassigned (value is unknown), the process proceeds to the activity named ChooseDepartment , which is a question activity (as indicated by the top left icon in the box), also known as DM, or dialog module .
- ❖ In its simplest form, a DM plays a prompt and activates the speech recognition engine with a given grammar or a set of grammars.
- ❖ However, a DM needs to deal with several discourse issues, such as timeouts, repromptings, and confirmations.
- ❖ So, the proper configuration of a DM requires the designer to set several functional parameters.



Lecture-#23

Natural Language Processing

A8706

UNIT-V

Text Summarization and Information Retrieval

Information Retrieval

- ❖ Information retrieval deals with the representation, storage, organization of, and access to information items.
- ❖ IR systems with suitable representation and organization of the information items allow users with a specific information need to access the information they are interested in.
- ❖ Users manifest their information need in the form of a query, usually a bag of keywords, to convey the need to the IR system.
- ❖ The IR system then retrieves items it believes are relevant to the user's information need.
- ❖ The goal of an IR system is to deliver a list of documents ranked by relevance to the user's query.
- ❖ The target of an IR system is to rank the relevant documents as high as possible and the non-relevant as low as possible.

Information Retrieval

Document Preprocessing:

- ❖ Preprocessing takes a set of raw documents as input and produces a set of **tokens** as output.
- ❖ Tokens are specific occurrences of terms (types) in a document that represent the smallest unit of meaning.
- ❖ This output defines the **vocabulary** that can be used to index a collection of documents.

Document Preprocessing:

Algorithm 11–1. The preprocessing pipeline of document d that results in a set of tokens T

1. $d \leftarrow \text{INPUT}$

2. $T \leftarrow \emptyset$

3. $[c_1, c_2, \dots] \leftarrow \text{character-stream}(d)$

4.

$B \leftarrow \text{tokenize}([c_1, c_2, \dots])$

5.while $B \neq \emptyset$ **do**

6. $t \leftarrow \text{POLL}(B)$

7.if $\text{is-compound}(t)$ **then**

8. $B \leftarrow B \cup \text{compound-split}(t)$

9.end if

10.if not $\text{is-stop-word}(t)$ **then**

11. $t = \text{normalize}(t)$

12. $T \leftarrow T \cup \{t\}$

13.end if end while return T

Information Retrieval

Document Syntax

- ❖ The content of documents is usually encoded in a syntax specific for the given **file type**.
- ❖ Based on the file type specification, the textual content of a document has to be extracted before indexing, avoiding, in particular, that vocabulary elements containing formatting instructions or metadata information are indexed.
- ❖ Examples for file types that require content extraction are PDF files or web pages. For both cases, many libraries are available that parse PDF or HTML files and extract the textual content.

Encoding and Script

- ❖ Encoding is the representation of letters through a number of bytes in a computer system.
- ❖ Historically, the ASCII character encoding scheme was widely used to encode English documents. Because this scheme is mainly limited to the Latin alphabet, it cannot be used for scripts having different letters.

Monolingual Information Retrieval

- ❖ MLIR can be seen as the problem of aggregating the results of IR systems in different languages.
- ❖ Apart from aggregation, language-specific preprocessing of queries is needed, particularly translation.
- ❖ In general, MLIR is based on the same index structures and relies on similar document and retrieval models as known from monolingual information retrieval.
- ❖ monolingual information retrieval, including
 - a. document representation
 - b. index structures
 - c. retrieval models
 - d. document a priori models

Document Representation

- ❖ The preprocessing of documents results in **token stream** representations of documents.
- ❖ The tokens are instances of terms, which are defined by words, stems or lemmas of words, or character n -grams.
- ❖ The IR models presented in this chapter are independent of the used vocabulary and can be applied to any term model.
- ❖ This means that occurrences of terms in documents are assumed to be independent of the occurrences of other terms in the same document.
- ❖ Given the independence assumption, documents can be represented using the **vector space model**.
- ❖ This vector space is spanned by the vocabulary in such a way that each dimension corresponds to a specific term.

Document Representation

- ❖ Documents are represented as vectors by a mapping function f , which maps token streams of documents d to term vectors d .
- ❖ Different functions f are used in literature, the most prominent being
 - ❖ **Boolean document model:** The value of a term dimension is set to 1 if the term occurs at least once in the document, otherwise to 0. BoW model
 - ❖ **TF document model:** The value of each dimension depends on the number of occurrences of terms in the document token stream, that is, the **term frequency**.
 - ❖ The term frequency can be directly used as value in the term vector. Variants are, for example, the normalization of the term frequency by document length.

Bag of Words Model

- ❖ **Bag of Words (BoW) model** is a simple algorithm used in **Natural Language Processing**.
- ❖ In **BoW model** a sentence or a document is considered as a '**Bag**' containing **words**.
- ❖ It will take into account the **words** and their frequency of occurrence in the sentence or the document disregarding semantic relationship in the sentences.

Feature Extraction

Example of the Bag-of-Words Model

The Bag of Words Representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

Feature Extraction

Example of the Bag-of-Words Model

Step 1: Collect Data

*It was the best of times,
it was the worst of times,
it was the age of wisdom,
it was the age of foolishness,*

For this small example, let's treat each line as a separate "document" and the 4 lines as our entire corpus of documents.

Feature Extraction

Step 2: Design the Vocabulary

Now we can make a list of all of the words in our model vocabulary.

The unique words here (ignoring case and punctuation) are:

“it”

“was”

“the”

“best”

“of”

“times”

“worst”

“age”

“wisdom”

“foolishness”

That is a vocabulary of 10 words from a corpus containing 24 words.

Feature Extraction

Step 3: Create Document Vectors

- ❖ The next step is to score the words in each document.
- ❖ The objective is to turn each document of free text into a vector that we can use as input or output for a machine learning model.
- ❖ Because we know the vocabulary has 10 words, we can use a fixed-length document representation of 10, with one position in the vector to score each word.
- ❖ The simplest scoring method is to mark the presence of words as a boolean value, 0 for absent, 1 for present.
- ❖ Using the arbitrary ordering of words listed above in our vocabulary, we can step through the first document (*"It was the best of times"*) and convert it into a binary vector.

Feature Extraction

The scoring of the document would look as follows:

- “it” = 1
- “was” = 1
- “the” = 1
- “best” = 1
- “of” = 1
- “times” = 1
- “worst” = 0
- “age” = 0
- “wisdom” = 0
- “foolishness” = 0

As a binary vector, this would look as follows:

1 [1, 1, 1, 1, 1, 1, 0, 0, 0, 0]

Feature Extraction

- 1 "it was the worst of times" = [1, 1, 1, 0, 1, 1, 1, 0, 0, 0]
- 2 "it was the age of wisdom" = [1, 1, 1, 0, 1, 0, 0, 1, 1, 0]
- 3 "it was the age of foolishness" = [1, 1, 1, 0, 1, 0, 0, 1, 0, 1]

		it		was		the		best		worst		times		age		wisdom		foolishness
Doc 1		1		1		1		1		0		1		0		0		0
Doc 2		1		1		1		0		1		1		0		0		0
Doc 3		1		1		1		0		0		0		1		1		0
Doc 4		1		1		1		0		0		0		1		0		1

Document Representation

- ❖ *TF.IDF document model*: These models additionally multiply term frequency values by the **inverse document frequency** of terms.
- ❖ The document frequency of a term is the number of documents in the collection containing this term.
- ❖ The inverse document frequency therefore puts more weight on infrequent terms and less weight on frequent terms that do not discriminate well between documents in the collection.
- ❖ In most cases, the logarithm of the inverse document frequency is used in TF.IDF models.

TF-IDF Model

- ❖ The presence of frequent word vectors won't be sparse (especially stopwords), though the word may not be important. Also, rare words look very sparse, hence less importance. To tackle these problems, we use TF-IDF vectorization.
- ❖ The first part is **TF**, called term frequency. This simply means the number of times the word occurs in a document divided by the total number of words in the document.
- ❖ The second part is **IDF**, which stands for 'inverse document frequency', interpreted like the inverted number of documents, in which the term we're interested in occurs.

Feature Extraction

Term Frequency

The term is frequency measure of a word w in a document (text) d . It is equal to the number of instances of word w in document d divided by the total number of words in document d . Term frequency serves as a metric to determine a word's occurrence in a document as compared to the total number of words in a document. The denominator is always the same.

$$\text{Term Frequency} = \frac{\text{number of instances of word } w \text{ in document } d}{\text{total number of words in document } d}$$

Inverse Document Frequency (IDF)

This parameter gives a numeric value of the importance of a word. Inverse Document frequency of word w is defined as the total number of documents (N) in a text corpus D , divided by the number of documents containing w .

$$IDF = \log \left(\frac{\text{total number of documents } (N) \text{ in text corpus } D}{\text{number of documents containing } w} \right)$$

Term Frequency Inverse Document Frequency (TF-IDF)

- ❖ The product of TF and IDF is the TF-IDF.
- ❖ TF-IDF is usually one of the best metrics to determine if a term is significant to a text.
- ❖ It represents the importance of a word in a particular document.

Feature Extraction

Example of 3 documents -

Document 1 It is going to rain today.

Document 2 Today I am not going outside.

Document 3 I am going to watch the season premiere.

To find TF-IDF we need to perform the steps we laid out above, let's get to it.

Word	Count
going	3
to	2
today	2
i	2
am	2
it	1
is	1
rain	1



Words/ Documents	Document 1
going	0.16
to	0.16
today	0.16
i	0
am	0
it	0.16
is	0.16
rain	0.16

Feature Extraction

Example of 3 documents -

Document 1 It is going to rain today.

Document 2 Today I am not going outside.

Document 3 I am going to watch the season premiere.

To find TF-IDF we need to perform the steps we laid out above, let's get to it.

Step 1 Clean data and Tokenize

Word	Count
going	3
to	2
today	2
i	2
am	2
it	1
is	1
rain	1



Step 2 Find TF of Document 1—

Words/ Documents	Document 1
going	0.16
to	0.16
today	0.16
i	0
am	0
it	0.16
is	0.16
rain	0.16

Feature Extraction

Step 2 Find TF of Document 1, 2, 3

Words/ Documents	Document 1	Document 2	Document 3
going	0.16	0.16	0.12
to	0.16	0	0.12
today	0.16	0.16	0
i	0	0.16	0.12
am	0	0.16	0.12
it	0.16	0	0
is	0.16	0	0
rain	0.16	0	0

Feature Extraction

Step 3 Find IDF

Words	IDF Value
going	$\log(3/3)$
to	$\log(3/2)$
today	$\log(3/2)$
i	$\log(3/2)$
am	$\log(3/2)$
It	$\log(3/1)$
is	$\log(3/1)$
rain	$\log(3/1)$

Feature Extraction

Step 4 Build model i.e. stack all words next to each other —

Words	IDF Value		Words/ Documents	Document 1	Document 2	Document 3
going	0		going	0.16	0.16	0.12
to	0.41		to	0.16	0	0.12
today	0.41		today	0.16	0.16	0
i	0.41		i	0	0.16	0.12
am	0.41		am	0	0.16	0.12
It	1.09		it	0.16	0	0
is	1.09		is	0.16	0	0
rain	1.09		rain	0.16	0	0

Feature Extraction

Step 5 Compare results and use table to ask questions

Remember, the final equation = $TF\text{-}IDF = TF * IDF$

Words/ Documents	going	to	today	i	am	it	is	rain
Document 1	0	0.07	0.07	0	0	0.17	0.17	0.17
Document 2	0	0	0.07	0.07	0.07	0	0	0
Document 3	0	0.05	0	0.05	0.05	0	0	0

Document Representation

- ❖ Given a collection of documents, the document term vectors can be aligned to form the **term-document matrix**.
- ❖ This matrix is spanned by terms as rows and documents as columns. We illustrate the different document representations using the following documents:

Doc1: It is a sunny day in Karlsruhe.

Doc2: It rains and rains and rains the whole day.

Document Representation

<i>Term</i>	<i>Boolean</i>		<i>TF</i>		<i>TF.IDF</i>	
	Doc1	Doc2	Doc1	Doc2	Doc1	Doc2
sunny	1	0	1	0	$1 \log 2/1 = 0.7$	0.0
day	1	1	1	1	$1 \log 2/2 = 0.0$	$1 \log 2/2 = 0.0$
Karlsruhe	1	0	1	0	$1 \log 2/1 = 0.7$	0.0
rains	0	1	0	3	0.0	$3 \log 2/1 = 2.1$

Index Structures

- ❖ The simplistic approach to scan through all documents given a query obviously does not scale to large collections.
- ❖ The high time performance of current retrieval systems is achieved by using an **inverted index**.
- ❖ The idea is to store for each term the information in which documents it occurs. This relation from terms to documents is called **posting list**.

For the preceding example documents, we get the following posting lists:

sunny -> doc1(1x)

day -> doc1(1x), doc2(2x)

Karlsruhe -> doc1(1x)

rains -> doc2(3x)

Retrieval Models

- ❖ Retrieval models are used to estimate relevance of documents to queries. Different theoretical models have been used to derive these relevance functions.
- ❖ In the following we describe three main families of retrieval models:
 - a. **boolean models**
 - b. **vector space models**
 - c. **probabilistic models.**
- ❖ Depending on the retrieval model, queries are represented in different ways.
- ❖ Boolean queries used for boolean models are modeled as a binary term vector.

Retrieval Models

Boolean Models

- ❖ Boolean models were the first retrieval models used in the beginning of information retrieval.
- ❖ In the case of the Boolean re-trieval model, relevance is binary and is computed by matching binary vectors representing term occurrence in the query to binary document vectors representing term occurrence.
- ❖ Because current vector space or probabilistic models outperform boolean models.

Retrieval Models

Vector Space Models

- ❖ Vector space models are based on vector space representations of documents.
- ❖ As described earlier, this vector space is spanned by the vocabulary, and entries in the term-document matrix are usually defined by term frequencies.
- ❖ There are different models to assess the relevance of documents to a given query:
Accumulative model:
 - ❖ The retrieval function computes scores for each query term.
 - ❖ The query term scores are summed up per document to get a final accumulated score for each document.
 - ❖ Functions computing scores for a single query term t are based on the following measures:

Retrieval Models

$tf_d(t)$. Term frequency in the document.

$|d|$. Length of the document.

$df(t)$. Document frequency of the query term.

$tf_D(t)$. Number of tokens of the query term in the whole collection.

$|D|$. Number of documents in the collection.

For example, the accumulated score of a simple retrieval model based on term frequency and inverse document frequency is computed as follows:

$$\text{score}(q, d) = \sum_{t \in q} tf_d(t) \log \frac{|D|}{df(t)}$$

Retrieval Models

Geometric model:

- ❖ The vector space representation of the query q can be interpreted as term vector .
- ❖ In this case, geometric similarity measures in the term vector space can be used as retrieval models.
- ❖ For example, the **cosine similarity** has been applied successfully in retrieval scenarios:

$$\text{score}(q, d) = \text{cosine}(\vec{q}, \vec{d}) = \frac{\langle \vec{q}, \vec{d} \rangle}{||\vec{q}|| ||\vec{d}||}$$

Probabilistic Models

In probabilistic retrieval models, the basic idea is to estimate the likelihood that documents are relevant to a given query.

Query Expansion

- ❖ The query is expanded by additional terms that further characterize the information need.
- ❖ The goal is to match more relevant documents that contain relevant content but use other terms to describe it.
- ❖ Expanded queries can be used in all retrieval models presented previously.
- ❖ Usually, expanded query terms are given less weight than the original query terms.
- ❖ The weight depends on the confidence of each expanded term and the overall weight put into query expansion.
- ❖ Using probabilistic retrieval models, query expansion can be used to improve the estimation of probabilities, such as the estimation of the query language model.
- ❖ We distinguish two different sources for expansion terms:
 - ❖ **Background Knowledge**
 - ❖ **Relevance Feedback**

Query Expansion

Background Knowledge

- ❖ Additional knowledge sources are exploited to find expansion terms for a given query.
- ❖ An example is the use of a the- saurus to expand the query with synonyms of the query terms.
- ❖ For CLIR or MLIR, a special case of query expansion is the translation of the query.
- ❖ In this case the query is expanded using the terms of its translation into different languages.

Relevance Feedback

- ❖ Using **relevance feedback** for query expansion is a two-step retrieval process.
- ❖ First, the original query is matched to the doc- ument collection.
- ❖ Then, relevance assessments are used to identify the relevant documents in the retrieval results.
- ❖ Using an expansion model based on term frequency and document frequency in this set of **expansion documents**, promising terms are identified and used in a second re- trieval step for query expansion.

Query Expansion

- ❖ The selection of relevant documents in the first step can be either manual or automatic.
- ❖ In the first case the user selects relevant documents manually out of the retrieval results of the first step.
- ❖ In the case of **pseudo-relevance feedback** (PRF), the top k documents of the first retrieval step are assumed to be relevant. This enables to implement automatic query expansion without user interaction.
- ❖ For this reason, PRF is often referred to as **blind relevance feedback**.

Document A Priori Models

- ❖ The a priori probability of documents is assumed to be uniform; that is, the probability of retrieving documents independently of a specific query is the same for all documents.
- ❖ However, in many scenarios this assumption does not hold.
- ❖ For example, documents have different perceived quality and popularity.
- ❖ Such factors could definitely influence the a priori probability of a document in the sense that very popular or high-quality documents should intuitively have a higher likelihood of being relevant.
- ❖ For probabilistic and language models, the estimation of document priors $P(d)$ is required as part of the retrieval model.
- ❖ In the standard case without background knowledge, document priors are assumed to have equal distribution over all documents.
- ❖ However, if document a priori models are available, they can be integrated directly into the retrieval model by replacing the uniform model used before.



Lecture-#24

Natural Language Processing

A8706

UNIT-V

Text Summarization and Information Retrieval

Inverted Index

- An inverted index is a special data structure used in search engines and information retrieval systems.
- Instead of mapping documents to the words they contain (like a book's contents), it maps each word (term) to a list of all the documents where that word appears.
- This makes it much faster to find all documents containing a search term, because the system doesn't need to scan every document from start to finish.

Inverted Index

Example

Imagine you have the following three documents:

Doc 1: "Cat chases dog"

Doc 2: "Dog bites man"

Doc 3: "Dog saves cat"

To build an inverted index:

Identify all unique terms: cat, chases, dog, bites, man, saves

Map each term to the documents it appears in:

Inverted Index

Map each term to the documents it appears in:

Term	Documents
cat	1, 3
chases	1
dog	1, 2, 3
bites	2
man	2
saves	3

Inverted Index

Map each term to the documents it appears in:

Term	Documents
cat	1, 3
chases	1
dog	1, 2, 3
bites	2
man	2
saves	3

Now, if you want to find all documents containing "cat", you can instantly retrieve Doc 1 and Doc 3 without scanning every document.

Inverted Index

Example: Search Engine Documents

Suppose there are three documents:

- Document 1: "Database indexing improves performance."
- Document 2: "Inverted indexes are crucial for search engines."
- Document 3: "Performance tuning is essential for databases."

Inverted Index

Term	Documents
database	1, 3
indexing	1
improves	1
performance	1, 3
inverted	2
indexes	2
crucial	2
search	2
engines	2
tuning	3
essential	3

Efficiency of Automated Document Summarization

How Summarization Works

- Extractive summarization selects and assembles key sentences or phrases directly from the original text, preserving essential information while reducing length.
- Abstractive summarization generates new sentences that capture the core ideas of the document, often paraphrasing and reorganizing content for clarity and coherence.

Benefits for Understanding Large Texts

- Summarization saves time by allowing users to quickly grasp the main ideas without reading the entire document.
- It improves clarity by presenting information in a compact and coherent format, making decision-making and content management more efficient.
- Automated summarization is especially valuable for news aggregation, legal documents, academic research, and financial reports, where large volumes of information must be processed rapidly.

Feature Matrix

A feature matrix in Natural Language Processing (NLP) is a structured, numerical representation of text data that allows machine learning algorithms to process and analyze language. Since algorithms can't work directly with raw text, we convert text into a matrix where:

- Rows represent documents (or sentences, reviews, etc.)
- Columns represent features (often words, n-grams, or other linguistic properties)
- Values indicate the presence, frequency, or importance of a feature in a document

Why Use a Feature Matrix?

- It transforms unstructured text into structured, numerical data.
- Enables algorithms to perform tasks like classification, clustering, or sentiment analysis

Feature Matrix

Example: Bag-of-Words Feature Matrix

Suppose you have three short documents:

1. "I love NLP"
2. "NLP is fun"
3. "I love fun"

Step 1: Build the Vocabulary

- Unique words: I, love, NLP, is, fun

In the given Matrix,

- Each row is a document.
- Each column is a word from the vocabulary.
- The numbers show how many times each word appears in each document.

Step 2: Create the Matrix

Document	I	love	NLP	is	fun
Doc 1	1	1	1	0	0
Doc 2	0	0	1	1	1
Doc 3	1	1	0	0	1

Relevance Feedback in Information Retrieval

Relevance feedback is an interactive process used in information retrieval systems (like search engines) where the user provides feedback on the search results—typically by marking which documents are relevant and which are not. The system then uses this feedback to refine the initial search query or reorder the results, with the goal of presenting more relevant documents in subsequent searches.

Steps Involved

1. Initial Query & Results:

The user submits a query; the system returns an initial list of ranked documents.

2. User Feedback:

The user reviews some results and indicates relevance (for example, by marking documents as "relevant" or "not relevant").

3. Query/Model Update:

The system analyzes the feedback to update either the query representation (by adding, removing, or reweighting terms) or adjusts the ranking model.

4. Refined Results:

The updated query/model retrieves or re-ranks documents, presenting improved results to the user.

Main Steps in Building a Question Answering System (NLP)

Question Understanding

- The system first reads and analyzes the user's question.
- It figures out what type of answer is needed (a fact, a list, an explanation, etc.) and identifies key words or phrases.
- Techniques used: tokenization (splitting text into words), part-of-speech tagging (identifying nouns, verbs, etc.), named entity recognition (finding names, places, dates), and parsing (understanding sentence structure).

Information Retrieval

- The system searches a large collection of documents or a database to find relevant information that might contain the answer.
- It uses methods like keyword matching, semantic search, and ranking algorithms to pick the most promising texts.
- Tools like inverted indexes help make this search fast and efficient.

Main Steps in Building a Question Answering System (NLP)

Answer Extraction or Generation

- From the retrieved texts, the system either extracts the exact answer (extractive QA) or generates a new answer in natural language (generative QA).
- Extractive QA finds the specific span of text that answers the question.
- Generative QA uses models to write a new answer based on the information found.

Answer Ranking and Presentation

- The system may find several possible answers, so it ranks them by relevance and accuracy.
- Finally, it presents the best answer to the user in clear, natural language.

Extractive vs. Abstractive Summarization

Extractive and abstractive text summarization are two major approaches in natural language processing for condensing information, differing fundamentally in how summaries are produced and the type of language they generate.

Extractive Summarization

- Extractive summarization works by identifying and selecting the most important sentences or phrases from the original text and joining them together, essentially copying verbatim from the source content without any modification or rewording.

Example:

Original text:

“Machine learning is a subset of artificial intelligence. It enables computers to learn from data and make predictions.”

Extractive summary:

“Machine learning is a subset of artificial intelligence. It enables computers to learn from data and make predictions.”

Both sentences are directly lifted from the original, resulting in a coherent but unaltered summary.

Extractive vs. Abstractive Summarization

Abstractive Summarization

- Abstractive summarization goes beyond simply copying parts of the original; it interprets, compresses, and rewrites information to generate new sentences that capture the main ideas of the source. The summary often uses rephrased text not found directly in the original, resembling how a human would summarize a passage.

Example:

Original text:

“Machine learning is a subset of artificial intelligence. It enables computers to learn from data and make predictions.”

Abstractive summary:

“Machine learning helps computers use data to make predictions, as part of AI.”

This sentence paraphrases and condenses the original while retaining key information, producing a more concise and often more readable summary.

Extractive vs. Abstractive Summarization

Aspect	Extractive Summarization	Abstractive Summarization
Method	Copies exact phrases/sentences from the source	Paraphrases and generates new sentences
Output language	Original, unchanged from input	May use new vocabulary and structure
Example summary	Directly pulled sentences	Rewritten, possibly shorter sentences
Reliability	Less risk of introducing errors/hallucinations	May introduce factual or logical errors
Complexity	Simpler, often rule-based	Requires advanced AI models

Blind relevance feedback

Also known as pseudo relevance feedback, is a technique used in information retrieval systems like search engines to automatically improve the quality of search results without requiring explicit feedback from users.

In blind relevance feedback:

- The system first executes the user's original query and retrieves a ranked list of documents.
- It then assumes that the top “k” results are relevant, even though the user has given no confirmation of relevance.
- The system analyzes these top results to identify the most significant terms (often using metrics like tf-idf) and expands the original query with these terms.
- A new search is conducted using the expanded query, and the results are returned to the user.

How It Improves Search Results

- By enriching the original query with additional terms found in the presumed relevant documents, the search engine can capture related concepts and vocabulary that might not have been included in the user's initial query.
- This can help retrieve documents that are relevant but would not have matched the original, possibly too narrow, query.
- Overall, blind relevance feedback can lead to better recall (finding more relevant documents) and sometimes improved precision in search result quality.
- However, if the top results are not actually relevant, the process may introduce unrelated terms and cause “query drift,” potentially reducing the result quality.

Single –document and Multiple- document Text summarization

Single-Document Summarization

- Focuses on extracting or generating the most important points from one source of text.
- The summary reflects only the ideas and contents of that single document, emphasizing clarity and coherence based on its structure and vocabulary.
- There is minimal concern for redundancy since the information comes from one coherent source.
- Example tasks: summarizing a news article, research paper, or report.

Multiple-Document Summarization

- Integrates information from two or more documents that share a related topic or theme.
- The system must detect, merge, and organize overlapping as well as unique content, often needing to handle conflicting perspectives or redundant facts.
- Methods for multi-document summarization must address redundancy elimination, passage selection, and thematic diversity, as documents can offer varying information and viewpoints on the same subject.
- Produces a summary that gives an overview across multiple sources, providing broader coverage and diversity of viewpoints.
- Example tasks: summarizing multiple news articles covering the same event, aggregating findings across various scientific papers.

Single –document and Multiple- document Text summarization

Aspect	Single-Document Summarization	Multiple-Document Summarization
Input scope	One document	Two or more related documents
Redundancy	Minimal or none	High, needs active elimination
Diversity	Single perspective	Multiple perspectives, thematic diversity
Complexity	Less complex, direct extraction	More complex, requires integration
Output coverage	Limited to source document	Broader, more comprehensive summary

Question Answering Types

1. Fact-based (Factual) Questions

- These ask for specific facts such as names, dates, places, or quantities.
- Example: “Who is the president of the United States?”
- Answer: “Donald Trump.”

2. List Questions

- Require the system to generate a list of items related to the query.
- Example: “List the planets in the solar system.”
- Answer: “Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune.”

3. Definition Questions

- Seek definitions or explanations of terms or concepts.
- Example: “What is photosynthesis?”
- Answer: “Photosynthesis is the process by which green plants use sunlight to synthesize food from carbon dioxide and water.”

Question Answering Types

4. How/Why Questions (Descriptive, Explanatory)

- Ask for procedural, causal, or descriptive explanations.
- Example (How): “How does rain form?”
- Answer: “Rain forms when water vapor in the air cools and condenses into droplets that fall due to gravity.”
- Example (Why): “Why do leaves change color in autumn?”
- Answer: “Leaves change color due to the breakdown of chlorophyll, revealing other pigments.”

5. Hypothetical Questions

- Inquire about possible scenarios or outcomes.
- Example: “What would happen if Earth stopped rotating?”
- Answer: “If Earth stopped rotating, there would be drastic changes in climate and gravity distribution.”

6. Coreference and Contextual Questions

- Require the QA system to understand context or references within a text or conversation.
- Example: “Where did Einstein win the Nobel Prize?” (after a passage about Einstein)
- Answer: “In Sweden.”

Question Answering Types

7. Mathematical and Computation-based Questions

- Require calculation and returning a numerical or formulaic answer.
- Example: “What is the area of a circle with radius 2?”
- Answer: “The area is $\pi \times 2^2 = 4\pi$.”

Collocations

- In Natural Language Processing (NLP), a *collocation* refers to a pair or group of words that frequently occur together more often than would be expected by chance and form a natural-sounding combination in a language.
- Collocations are important because they reflect authentic language use, helping applications like machine translation, language modeling, and text analysis produce more native-like and accurate output.
- **Simple Examples of Collocation Usage**
 - **Strong tea:**

In English, people say “strong tea” rather than “powerful tea,” even though both “strong” and “powerful” are synonyms. This habitual pairing of “strong” and “tea” is a classic collocation.
 - **Make a decision:**

The verb “make” is typically used with “decision,” rather than “do a decision.” The phrase “make a decision” is considered a collocation because it naturally occurs together in English usage.
 - **Fast food:**

The words “fast” and “food” commonly appear together in the context of convenient eatery options. Other alternatives like “quick food” are rarely used, so “fast food” is a collocation.

TF-IDF (Term Frequency–Inverse Document Frequency)

- TF-IDF (Term Frequency–Inverse Document Frequency) is a statistical method used in natural language processing to assess the importance of a word in a document within a corpus.
- **Term Frequency (TF)** measures how often a word appears in a particular document, relative to the document's length.
- **Inverse Document Frequency (IDF)** measures how unique or informative a word is across all documents in the corpus. Words that appear in many documents get a low score, while rare words score higher.
- The TF-IDF value for each term is calculated as:
 - **$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t,D)$**
- TF-IDF helps to highlight terms that are frequent in a document but not common across the corpus, making them useful for understanding or comparing documents.

TF-IDF (Term Frequency–Inverse Document Frequency)

Calculating the TF-IDF Vector with Stop Words

- Given corpus:

Doc1: "I am happy today"

Doc2: "I am unhappy today"

Doc3: "All are happy today"

Let's walk step by step, keeping ALL stop words (like "I," "am," etc.):

Step 1: List All Unique Words

I, am, happy, today, unhappy, all, are

Step 2: Compute TF (Term Frequency)

For each word in each document:

TF = (Number of times term occurs in the document) / (Total words in the document)

Doc1: I(1/4), am(1/4), happy(1/4), today(1/4)

Doc2: I(1/4), am(1/4), unhappy(1/4), today(1/4)

Doc3: all(1/4), are(1/4), happy(1/4), today(1/4)

TF-IDF (Term Frequency–Inverse Document Frequency)

Step 3: Compute IDF (Inverse Document Frequency)

$$\text{IDF} = \log(N/\text{DF})$$

N = total number of documents = 3

DF = number of documents containing the term

Term	DF	IDF
I	2	$\log(3/2)=0.176$
am	2	0.176
happy	2	0.176
today	3	$\log(3/3)=0$
unhappy	1	$\log(3/1)=0.477$
all	1	0.477
are	1	0.477

TF-IDF (Term Frequency–Inverse Document Frequency)

Step 4: Compute TF-IDF for Each Word in Each Document

Multiply TF by corresponding IDF:

Doc1:

- I: $1/4 \times 0.176 = 0.044$
- am: 0.044
- happy: 0.044
- today: $1/4 \times 0 = 0$
- unhappy: 0
- all: 0
- are: 0

Doc2:

- I: 0.044
- am: 0.044
- unhappy: $1/4 \times 0.477 = 0.119$
- today: 0
- happy: 0
- all: 0
- are: 0

Doc3:

- all: 0.119
- are: 0.119
- happy: 0.044
- today: 0
- I: 0
- am: 0
- unhappy: 0

TF-IDF (Term Frequency–Inverse Document Frequency)

Step 5: TF-IDF Vectors

Presented in the order: [I, am, happy, today, unhappy, all, are]

- Doc1: [0.044, 0.044, 0.044, 0, 0, 0, 0]
- Doc2: [0.044, 0.044, 0, 0, 0.119, 0, 0]
- Doc3: [0, 0, 0.044, 0, 0, 0.119, 0.119]

These vectors represent the importance of each word in each document, even when stop words are included (many stop words get low or zero scores due to low IDF).

- The calculated TF-IDF vectors highlight which words are most informative in each document, even when stop words are included.
- Words that appear across all documents, like "today," have a TF-IDF score of zero, indicating they provide little unique information for distinguishing between documents.
- In contrast, words that are less frequent in the corpus, such as "unhappy," "all," and "are," receive higher scores, marking them as more relevant and useful for identifying and differentiating the documents.

Document-Term Matrix

- A document-term matrix (DTM) is a mathematical matrix used to represent the frequency of terms that appear in a collection of documents. In this matrix:
 - Each row corresponds to a document.
 - Each column corresponds to a unique term from the corpus.
 - Each cell indicates how many times that term appears in that document.

This representation, also known as the *bag-of-words* model, helps turn unstructured text into numerical data suitable for computational analysis and machine learning tasks like classification, clustering, and search.

Document-Term Matrix

Document-Term Matrix for the Given Corpus
Corpus:

- Doc1: "I like apple"
- Doc2: "I unlike apple"
- Doc3: "you like apple"
- Doc4: "you unlike apple"

Step 1: List All Unique Terms (Vocabulary)

- I
- you
- like
- unlike
- apple

Document-Term Matrix

Step 2: Fill in Term Counts for Each Document

Document	I	you	like	unlike	apple
Doc1	1	0	1	0	1
Doc2	1	0	0	1	1
Doc3	0	1	1	0	1
Doc4	0	1	0	1	1

The table shows how many times each unique term appears in every document in the corpus. This structure enables computers to analyze patterns and relationships in the text, such as document similarity or word importance

Automatic Text Summarization System

Key NLP Tasks for Automatic Text Summarization

- Developing an automatic text summarization system involves several core NLP tasks. Each task plays a specific role in transforming raw text into a concise, meaningful summary. Here's an analysis of the main tasks and their contributions:

1. Text Preprocessing

- What it does: Cleans and standardizes the input text (removing special characters, lowercasing, tokenizing, etc.).
- Contribution: Ensures the text is in a consistent format, making downstream processing more accurate and efficient. For example, removing noise helps models focus on meaningful content.

2. Sentence Segmentation

- What it does: Splits the text into individual sentences.
- Contribution: Summarization often works at the sentence level, especially for extractive methods. Accurate segmentation is crucial for identifying and ranking candidate sentences for the summary.

3. Tokenization

- What it does: Breaks sentences into words or tokens.
- Contribution: Enables further analysis like frequency counts, part-of-speech tagging, and embedding generation, which are foundational for both extractive and abstractive summarization.

Automatic Text Summarization System

4. Part-of-Speech (POS) Tagging & Syntactic Parsing

- What it does: Labels each word with its grammatical role and analyzes sentence structure.
- Contribution: Helps identify key phrases, subjects, and objects, which can be important for understanding sentence importance and for generating coherent summaries, especially in abstractive approaches.

5. Named Entity Recognition (NER)

- What it does: Identifies entities like people, places, organizations, and dates.
- Contribution: Ensures that summaries retain critical factual information and do not omit or misrepresent key entities.

6. Coreference Resolution

- What it does: Determines when different words refer to the same entity (e.g., "he" = "Dr. Smith").
- Contribution: Maintains clarity and coherence in summaries by resolving pronouns and references, which is especially important in abstractive summarization.

Automatic Text Summarization System

7. Sentence/Passage Scoring and Selection (Extractive Summarization)

- What it does: Ranks sentences by importance using features like term frequency, position, or graph-based algorithms (e.g., TextRank).
- Contribution: Selects the most relevant sentences to include in the summary, ensuring coverage of main ideas while reducing redundancy.

8. Semantic Understanding & Representation (Abstractive Summarization)

- What it does: Uses deep learning models (like transformers) to build contextual representations of the text.
- Contribution: Enables the system to paraphrase, compress, and generate new sentences that capture the essence of the original text, rather than just copying it.

9. Redundancy Removal & Coherence Checking

- What it does: Detects and removes repeated information; ensures logical flow.
- Contribution: Produces summaries that are concise, non-repetitive, and easy to read, especially when combining content from multiple parts of a document or multiple documents.

10. Post-processing

- What it does: Final clean-up, formatting, and sometimes length control.
- Contribution: Ensures the summary is grammatically correct, well-formatted, and meets any length or style requirements.

Bag-of-Words (BoW) Model

The Bag-of-Words (BoW) model is a simple and widely used method in Natural Language Processing (NLP) for representing text data as numerical vectors. In BoW:

- Each document is treated as a "bag" of its words, ignoring grammar and word order.
- The model builds a vocabulary of all unique words in the corpus.
- Each document is represented as a vector, where each element is the count (frequency) of a word from the vocabulary in that document.
- This approach allows text to be processed by machine learning algorithms, which require numerical input.

Bag-of-Words (BoW) Model

Step-by-Step: BoW Vector for the Given Corpus (with Stop Words)

Corpus:

- Doc1: "I am happy today"
- Doc2: "I am unhappy today"
- Doc3: "All are happy today"

1. List All Unique Words (Vocabulary)

From all documents, the unique words are:

- I
- am
- happy
- today
- unhappy
- all
- are

Bag-of-Words (BoW) Model

Build the Document-Term Matrix (BoW Vectors)

Document	I	am	happy	today	unhappy	all	are
Doc1	1	1	1	1	0	0	0
Doc2	1	1	0	1	1	0	0
Doc3	0	0	1	1	0	1	1

- Each row is a document.
- Each column is a word from the vocabulary.
- Each cell shows how many times that word appears in the document.

Bag-of-Words (BoW) Model Vs. TF IDF Vs. Document-Term Matrix Vs. Inverted Index

Aspect	Bag of Words (BoW)	TF-IDF	Document-Term Matrix	Inverted Index
Representation	Word counts	Weighted word importance	Matrix: docs $\times$ terms	Term $\rightarrow$ list of docs
Captures Word Order?	No	No	No	No
Handles Common Words	No	Yes (down-weights them)	Depends on weighting	N/A
Main Use	Simple text features	Feature selection, search	Data for ML, clustering	Fast search, retrieval

Question Answering Components

Main Components of a Question Answering (QA) System

- A typical QA system is designed to take a user's natural language question and return a precise, relevant answer. The architecture generally consists of three main components, each playing a crucial role in the process

1. Question Understanding (Analysis)

- Purpose: Analyze and interpret the user's question to determine its intent, type (e.g., fact, list, definition), and the information needed to answer it.
- Techniques Used: Tokenization, part-of-speech tagging, named entity recognition, dependency parsing, and question classification.
- Contribution: Ensures the system understands what the user is really asking, which guides the retrieval and answer generation steps.

2. Information Retrieval (Document/Passage Retrieval)

- Purpose: Search a large knowledge source (like a database, document collection, or the web) to find relevant documents or passages that may contain the answer.
- Techniques Used: Keyword matching, semantic search, document ranking, and sometimes vector-based retrieval (e.g., using TF-IDF or neural embeddings).
- Contribution: Narrows down the vast information space to a manageable set of candidate texts likely to contain the answer.

Question Answering Components

3. Answer Extraction or Generation

- Purpose: From the retrieved texts, extract the exact answer (extractive) or generate a new answer (abstractive) in natural language.
- Techniques Used: Sentence selection, answer ranking, span extraction (for extractive QA), or natural language generation (for abstractive QA, often using large language models).
- Contribution: Delivers a concise, accurate, and well-formed answer to the user.

How the System Works: Example Walkthrough User Query: "Who invented the telephone?"

1. Question Understanding:

The system identifies this as a fact-based question about a person ("Who").

2. Information Retrieval:

It searches its knowledge base or document collection for passages mentioning the invention of the telephone.

3. Answer Extraction/Generation:

From the relevant passages, it extracts or generates the answer: "Alexander Graham Bell."

TF-IDF vector (with stop words included)

Find the TF-IDF vector with stop words from given corpus:

Doc1: "Banana is a fruit"

Doc2: "Apple is a fruit"

Doc3: "Orange and Grape are fruit"

Find the feature document vector for the document

Doc4: "Apple and orange are fruit"

Step 1: List All Unique Words (Vocabulary)

From all documents (Doc1, Doc2, Doc3, Doc4):

- banana
- is
- a
- fruit
- apple
- orange
- and
- grape
- are

TF-IDF vector (with stop words included)

Step 2: Compute Term Frequency (TF)

For each document, $TF = (\text{number of times term appears in document}) / (\text{total words in document})$.

Doc1: "Banana is a fruit" (4 words)

- banana: $1/4$
- is: $1/4$
- a: $1/4$
- fruit: $1/4$
- others: 0

TF-IDF vector (with stop words included)

Doc2: "Apple is a fruit" (4 words)

apple: $\frac{1}{4}$

- is: $\frac{1}{4}$
- a: $\frac{1}{4}$
- fruit: $\frac{1}{4}$
- others: 0

Doc3: "Orange and Grape are fruit" (5 words)

- orange: $\frac{1}{5}$
- and: $\frac{1}{5}$
- grape: $\frac{1}{5}$
- are: $\frac{1}{5}$
- fruit: $\frac{1}{5}$
- others: 0

Doc4: "Apple and orange are fruit" (5 words)

- apple: $\frac{1}{5}$
- and: $\frac{1}{5}$
- orange: $\frac{1}{5}$
- are: $\frac{1}{5}$
- fruit: $\frac{1}{5}$
- others: 0

TF-IDF vector (with stop words included)

Step 3: Compute Inverse Document Frequency (IDF)

$IDF = \log(N / DF)$,
where $N = 3$ (since Doc4 is the query, not in the corpus),

and

DF = number of documents containing the term.

TF-IDF vector (with stop words included)

Term	DF	IDF ($\log(3/DF)$)
banana	1	0.477
is	2	0.176
a	2	0.176
fruit	3	0
apple	1	0.477
orange	1	0.477
and	1	0.477
grape	1	0.477
are	1	0.477

TF-IDF vector (with stop words included)

Step 4: Compute TF-IDF for Doc4

Multiply TF by IDF for each term:

- banana: $0 \times 0.477 = 0$
- is: $0 \times 0.176 = 0$
- a: $0 \times 0.176 = 0$
- fruit: $1/5 \times 0 = 0$
- apple: $1/5 \times 0.477 = 0.095$
- orange: $1/5 \times 0.477 = 0.095$
- and: $1/5 \times 0.477 = 0.095$
- grape: $0 \times 0.477 = 0$
- are: $1/5 \times 0.477 = 0.095$

TF-IDF Feature Vector for Doc4

In the order: [banana, is, a, fruit, apple, orange, and, grape, are]

- Doc4: [0, 0, 0, 0, 0.095, 0.095, 0.095, 0, 0.095]

